Technical Architecture

Product (working title): EventArena [P] **Document status:** Draft v0.1 — architecture proposal. **Nothing described here is implemented or load-tested. Last updated / infrastructure facts verified:** 2026-09-25 **Companion documents:** PRD.md · RULES.md · DESIGN.md · TASKS.md · MEMORY.md

Evidence tags ([V] [S] [E] [T] [P] [U]) are defined in PRD.md. Business-rule IDs (BR-xx) and requirement IDs (FR-…, NFR-…) refer to PRD.md.

Table of contents

1. Purpose and scope
2. Repository inspection result
3. Architectural drivers
4. Infrastructure research
5. Technology selection and alternatives
6. System architecture
7. Data model
8. Authentication and authorization
9. Event, registration and round lifecycles
10. Round engine
11. Quiz engine
12. Development challenge submissions
13. Manual judging
14. Scoring and results
15. Leaderboard
16. CSV export
17. API design
18. Scalability analysis
19. Security
20. Reliability and failure handling
21. Deployment, environments and configuration
22. Monitoring
23. Backup and recovery
24. Testing and load-test strategy
25. Decision register
26. Extensibility and migration
27. Known limitations and open items

---

1. Purpose and scope

This document defines the proposed technical architecture for the platform described in PRD.md: a modular monolith that hosts many independent events, runs configurable rounds under server-authoritative timers and scoring, publishes access-controlled leaderboards, and exports safe CSV files — designed for 500+ simultaneous participants.

It separates **verified facts** (provider limits), **estimates** (workload model), and **targets** (to be tested). It deliberately does not assert that the free tiers or the design "will scale"; it identifies where they demonstrably cannot and what must be measured.

2. Repository inspection result

Item	Finding (2026-09-25)
Repository mounted in the working environment	**None found.** The workspace contained no `.git` directory, no source files, no `package.json`, no uploads, and no pre-existing `docs/` folder
Existing versions of the six documents	None existed; nothing was overwritten or deleted
Consequence	The project is treated as **greenfield**. Technology choices are proposals (`[P]`), not adaptations of existing code
Required follow-up	**T-0.1**: when the real repository is available, inspect its structure, dependencies, lint/test setup and any documentation; reconcile this architecture with it; record conflicts in the Decision register (§25) before writing code

If a repository exists elsewhere, RULES.md rule **R-REPO-01** requires the inspection to happen and these documents to be reconciled **before** any modification.

3. Architectural drivers

#	Driver	Architectural consequence
A1	500+ simultaneous users with synchronized surges	Short, single-transaction write paths; pooled connections; snapshot-based reads; measured headroom
A2	Server-authoritative timers/scores	Absolute deadlines in DB; DB clock; scoring in server code from immutable inputs
A3	Free-tier-first, verified limits	Free tiers for dev/staging; explicit paid "event tier"; no dependence on capabilities free tiers cap below 500
A4	Multi-event isolation	`event_id` on every event-scoped table, composite FKs, authorization scoping helpers, RLS defense in depth
A5	Reproducible, auditable results	Append-only score records, versioned configs, result runs, audit log
A6	Extensible round types	Plugin registry + typed contracts
A7	Single small team	Modular monolith, one deployable, boring technology
A8	Restart/deploy safety	No in-memory authority; stateless functions; DB-backed jobs

4. Infrastructure research

4.1 Method and date

All entries were checked on **2026-09-25** using web search and page fetches. `[V]` means the number appears on the provider's own pricing/documentation page retrieved that day. `[S]` means only a third-party page was seen (or a provider page could not be fetched in full). Provider pages change; every `[V]` item must be re-verified in task **T-0.5** before any purchase or event commitment. Nothing here is a quote.

4.2 Verified provider limits

Hosting / compute

Provider	Limit or behavior	Tag	Source
Vercel Hobby	Restricted to non-commercial personal use; commercial use requires Pro/Enterprise	[V]	https://vercel.com/docs/limits/fair-use-guidelines · https://vercel.com/docs/plans/hobby
Vercel (all plans)	Functions auto-scale to 30,000 concurrent on Hobby/Pro; burst limit 1,000 concurrent executions per 10 s per region; beyond → `503 FUNCTION_THROTTLED`	[V]	https://vercel.com/docs/functions/concurrency-scaling
Vercel Functions (Node)	Max duration Hobby 300 s (default and max); Pro default 300 s, up to 800 s; 2 GB memory on Hobby	[V]	https://vercel.com/docs/functions/limitations
Vercel Hobby	Cannot connect a Hobby project to repositories owned by Git *organizations* (personal repos are fine)	[V] (docs snippet)	https://vercel.com/docs/limits
Vercel Hobby	≈100 GB transfer, 1M edge requests, 1M function invocations, 4 CPU-hours, 100 deployments/day, 1 concurrent build per month/day; overage cannot be purchased — features pause	[S]	https://costbench.com/software/developer-tools/vercel/free-plan/ · https://vercel.com/docs/limits (verify numbers)
Vercel Pro	$20 per seat per month plus usage	[S]	https://temps.sh/blog/vercel-pricing-complete-guide-2026
Cloudflare Workers Free	100,000 requests/day; 10 ms CPU per request; 50 external subrequests; 128 MB; 5 Cron Triggers; static-asset requests free and unlimited	[V]	https://developers.cloudflare.com/workers/platform/limits/ · https://developers.cloudflare.com/workers/about/pricing
Cloudflare Workers Paid	$5/month minimum; 10M requests/month included then $0.30/M; up to 5 min CPU/request (30 s default)	[V]	https://developers.cloudflare.com/workers/about/pricing
Render Free web service	Spins down after 15 min idle; ~1 min spin-up; 750 free instance hours/workspace/month; ephemeral filesystem; may restart at any time; no outbound SMTP ports; free Postgres expires after 30 days	[V]	https://render.com/docs/free
Fly.io, Railway, Koyeb	Not researched from official sources; secondary sources suggest no permanent free tier or small credits	[U]	—

Relational databases

Provider	Limit or behavior	Tag	Source
Supabase Free	500 MB DB (shared CPU, 500 MB RAM); 5 GB egress; 1 GB file storage (50 MB max upload); 50,000 MAU; unlimited API requests; **no automatic backups**; **paused after 1 week of inactivity**; 2 active projects; 1-day log retention; community support; no uptime SLA	[V]	https://supabase.com/pricing.md (mirror of https://supabase.com/pricing)
Supabase Free	Custom SMTP included; basic MFA (TOTP) included	[V]	same
Supabase Free	Direct/pooler connection limits for the *Free* compute are not itemized on the pricing page (Micro is listed as 60 direct / 200 pooler)	[U] for Free	https://supabase.com/pricing.md
Supabase Pro	From $25/month/org; 8 GB disk; 250 GB egress; daily backups kept 7 days; never pauses; 100,000 MAU; spend cap on by default	[V]	same
Supabase compute add-ons	Micro $10 (shared, 1 GB, 60 direct / 200 pooler); Small $15 (shared, 2 GB, 90 / 400); Medium $60 (shared, 4 GB, 120 / 600); Large $110 (dedicated 2 vCPU, 8 GB, 160 / 800). Pro includes **$10/month compute credit** (covers one Micro)	[V]	same
Supabase PITR	$100/month per 7 days of retention	[V]	same
Supabase Auth email	Default built-in SMTP is heavily rate-limited (reported 2 emails/hour); after custom SMTP is set, a low initial limit (reported 30/hour) applies and is adjustable	[S]	https://supabase.com/docs/guides/auth/rate-limits (verify)
Neon Free	Per project: 100 CU-hours/month, 0.5 GB storage, autoscale to 2 CU, 5 GB transfer, 6-hour restore history; **scale-to-zero always on (5 min idle) and cannot be disabled**; when CU-hours are exhausted compute is suspended until next period	[V]	https://neon.com/docs/introduction/plans · https://neon.com/faqs/free-plan-limits-and-quotas
Neon Launch	Usage-based: $0.106/CU-hour, $0.35/GB-month; scale-to-zero can be disabled; history up to 7 days	[V]	same
Cloudflare D1 Free	5M rows read/day; 100,000 rows written/day (indexes add written rows); 5 GB; limits reset 00:00 UTC	[V]	https://developers.cloudflare.com/d1/platform/pricing/
Cloudflare D1 Free	From Sept 1, 2026 exceeding daily limits makes queries fail until reset	[S]	https://omidsaffari.com/blog/cloudflare-d1-free-limits-queries-stop
Cloudflare D1 Paid	25B rows read + 50M rows written/month included; $0.75/GB-month above 5 GB	[V]	https://developers.cloudflare.com/d1/platform/pricing/
Firestore (Spark/free quota)	1 GiB; 50,000 reads/day; 20,000 writes/day; 20,000 deletes/day; 10 GiB/month egress; resets midnight Pacific	[V]	https://firebase.google.com/docs/projects/billing/firebase-pricing-plans · https://cloud.google.com/firestore/quotas
Firebase Realtime Database (Spark)	100 simultaneous connections	[V]	https://firebase.google.com/docs/database/usage/limits

Realtime / messaging

Provider	Limit or behavior	Tag	Source
Supabase Realtime	Free: 200 concurrent connections, 100 msg/s, 2M msgs/month, 256 KB payload. Pro: 500 concurrent (then $10 per 1,000), 500 msg/s, 5M msgs/month included, 3 MB payload. Pro without spend cap: 10,000 connections. Channel joins/s: Free 100, Pro 500. Exceeding → `too_many_connections`/`too_many_joins`; client library reconnects	[V]	https://supabase.com/docs/guides/realtime/limits · https://supabase.com/pricing.md
Ably Free	200 concurrent connections, 200 channels, 6M messages/month, 500 msg/s; fan-out counts N+1 messages	[V]	https://ably.com/docs/platform/pricing · https://ably.com/pricing
Pusher Sandbox	100 concurrent connections; daily message cap	[S]	https://ably.com/compare/ably-vs-pusher/pricing
Cloudflare Durable Objects (Free)	SQLite-backed only; 100,000 requests/day; 13,000 GB-s/day; rows written 100,000/day; exceeding fails; WebSocket messages billed 20:1	[V]	https://developers.cloudflare.com/durable-objects/platform/pricing/
Vercel WebSockets	Connections close at function max duration; no sticky routing by room	[S]	https://github.com/endel/colyseus-vercel (unofficial)

Cache, rate limiting, email, load testing

Provider	Limit or behavior	Tag	Source
Upstash Redis Free	256 MB; 500,000 commands/month; 10 GB bandwidth; 10,000 cmd/s; 1 free DB. Pay-as-you-go $0.20 per 100K commands with budget cap	[V]	https://upstash.com/pricing/redis · https://upstash.com/docs/redis/overall/billing
Resend Free	3,000 emails/month, **100/day**; Pro $35/month for 100,000/month (an alternate third-party page says $20/50k — conflicting; verify)	[V]/[S]	https://resend.com/pricing.md · https://resend.com/docs/knowledge-base/what-is-resend-pricing.md
Brevo Free	300 email sends/day; provider footer branding on free	[V]	https://help.brevo.com/hc/en-us/articles/208580669
Grafana Cloud k6 Free	500 virtual-user-hours/month; per-test max VUs on Free not confirmed (an old community post says 100)	[V] (VUh) / [U] (VU cap)	https://grafana.com/products/cloud/performance-load-testing-k6/

Not yet researched (must be verified in T-0.5)

[U] GitHub Actions free minutes and scheduled-workflow behavior; Sentry free-tier limits; Vercel Cron limits on Hobby; Vercel Firewall/rate-limit availability per plan; Supabase `pg_cron`/`pg_net` minimum interval and availability; Supabase region list (Mumbai availability) and Vercel function region availability; Cloudflare Turnstile limits; uptime-monitor free plans; Supabase Storage CDN egress accounting.

4.3 Modeled demand vs. verified limits

Workload assumptions are in §18.1. [E] figures are estimates for **one** 500-participant event day (one quiz with 30 questions plus a challenge round, 3 hours of leaderboard viewing).

Limit (verified)	Modeled demand [E]	Assessment
Supabase Free Realtime: 200 concurrent	500 viewers if each holds a WebSocket	**Exceeds 2.5×** → participants must not depend on WebSockets; use snapshot polling (§15)
Firebase RTDB Spark: 100 connections	500	**Exceeds 5×**
Ably Free: 200 connections	500	**Exceeds 2.5×**
Firestore free: 20,000 writes/day	15,000 final answers + autosave upserts ≈ 45,000 writes	**Exceeds ≈ 2×**
D1 free: 100,000 rows written/day	≈ 45,000 table writes plus index-row writes ≈ 90,000–135,000 rows	**At or over the cap** with no headroom for other tables
Render free: 15-min spin-down, ~1-min cold start	A round starts after idle time	**Unacceptable** for a live start
Supabase Free: paused after 1 week idle; no backups	Event days follow long idle periods	**Unacceptable** as sole production DB
Vercel Hobby: non-commercial only [V]	Depends on event nature	**Approval item D-P02**
Vercel Hobby ≈1M function invocations/month [S]	≈ 350–450k per event day	≈ 35–45% of a month's allowance; two such days would approach the cap
Upstash Free: 500K commands/month	Rate-limit checks ≈ 3 commands × ≈ 20 API calls × 500 users ≈ 30k for a quiz	Fits — **only if leaderboard polling is not rate-limited through Redis**
Resend Free: 100 emails/day; Brevo Free: 300/day	500 verification emails	Resend: 5 days; Brevo: 2 days → verification must be optional, staged, or on a paid tier
Supabase Pro/Micro pooler: 200 client connections	Up to 500 concurrent invocations at a surge	Possible exhaustion → Small (400) or Medium (600) compute for events (see §18.4)

4.4 Can an all-free architecture support the workload?

It cannot be verified as adequate, and several documented properties make it unsuitable as the sole platform for a live 500-participant event:

1. No automated backups and inactivity pausing on Supabase Free [V] — an event day must never be the first request after a week of silence, and losing the only database is unrecoverable without our own dumps.
2. Free realtime tiers (Supabase 200, Ably 200, Firebase 100) are below 500 concurrent [V].
3. Vercel Hobby is non-commercial [V] and its request/invocation allowances leave little multi-event headroom [S].
4. Free email caps (100–300/day) cannot deliver 500 verification emails in a day [V].
5. No provider on a free plan offers an uptime SLA; free plans state community support only [V] (Supabase).
6. Capacity of shared-CPU free compute is not documented; it must be **measured** (LT-x in §24), not assumed.

Conclusion: use free tiers for development, CI, staging and functional testing. Treat the live-event configuration as a small, time-boxed paid **event tier** (§4.5). A free-tier live event is possible only if the owner explicitly accepts the risks above (D-P02, R-01, R-09) after load tests on that exact configuration.

4.5 Recommended low-cost event tier and cost drivers

Component	Dev/staging (free)	Event tier	List price basis
App hosting/functions	Vercel Hobby (if non-commercial)	Vercel Pro (1 seat)	$20/seat/month [S]
Postgres + Auth + Storage + Realtime	Supabase Free (2 projects: staging, dev)	Supabase Pro org, Small compute	$25 + $15 compute − $10 credit = **$30/month** [V] (org billing formula, Supabase pricing page)
Rate limiting	Upstash Free	Upstash pay-as-you-go with monthly budget cap	$0.20 per 100K commands [V]
Email	Brevo Free (300/day)	Brevo Free with staged verification, or Resend Pro	$0 or $35/month [V]
Monitoring / CI / load test	Free tiers [U] for Sentry/GitHub Actions; Grafana k6 Free 500 VUh [V]	same	$0
Backups	Own nightly dump	Supabase daily backups (7 days) + own dumps; PITR optional	PITR $100/month per 7 days [V] (optional)

Arithmetic (list prices, excluding tax and overages; not a quote): ≈ **$50/month** (Vercel Pro + Supabase Pro Small, Brevo free) to ≈ **$85/month** (adding Resend Pro). Because plans are monthly, the event tier can be enabled for the event month and downgraded afterwards [U — confirm proration and downgrade terms].

Main cost drivers, in order: (1) database compute size, because pooler/direct connection limits scale with compute [V]; (2) commercial-eligible app hosting; (3) email volume; (4) realtime connections if participants use WebSockets (Pro: $10 per additional 1,000 [V]); (5) optional PITR; (6) load-test virtual-user hours beyond the free 500.

4.6 Alternatives considered

Alternative	Verdict	Reason
All-Cloudflare (Workers + D1 + Durable Objects + Pages)	Strong low-cost contender; **not selected**	Workers Paid is only $5/month [V], but D1 is single-writer SQLite (no `SKIP LOCKED`/advisory locks/window-function-heavy SQL parity with Postgres), free D1 write cap is exceeded by one quiz [E], and the owner's stack experience is Next.js/PostgreSQL. Re-evaluate if Postgres cost or latency proves problematic
Firebase (Firestore + Auth + Hosting)	Rejected	Free write/connection quotas too low [V]; document model is poor for relational scoring/exports; pay-per-operation costs are unpredictable
Neon Postgres + separate auth/realtime	Viable substitute for the DB	Free plan: scale-to-zero cannot be disabled [V] → cold starts before rounds; Launch removes that. Kept as **fallback DB** because it is plain Postgres; would require replacing Supabase Auth/Storage/Realtime
Render web service + Render Postgres	Rejected for events	Free spin-down and 30-day DB expiry [V]; paid tiers not researched
Self-managed VPS (Docker, Postgres, Node, WebSocket server)	Rejected for MVP	Highest capacity per dollar potentially, but ops burden (patching, backups, TLS) and not free; pricing not researched [U]
Ably/Pusher for participant realtime	Not selected	Free caps (200/100) below 500 [V]; paid tiers add cost; polling design removes the need

4.7 Selected infrastructure (proposed [P])



5. Technology selection and alternatives

5.1 Selected primary architecture

A modular monolith: one Next.js (App Router) + TypeScript application deployed to Vercel, using Supabase-hosted PostgreSQL as the single system of record. Server logic lives in route handlers and server actions organized into domain modules with enforced boundaries. Background work

runs as DB-backed jobs advanced by authenticated "tick" endpoints. Leaderboards are versioned snapshots served over cached HTTP with an optional realtime "version ping".

Why this fits: relational integrity and transactions are central (registrations, attempts, scoring, audits); Postgres provides `SELECT … FOR UPDATE`, advisory locks, `ON CONFLICT`, window functions, `jsonb`, RLS, and `FOR UPDATE SKIP LOCKED`; serverless request-scoped compute suits bursty events; and the stack (Next.js, TypeScript, PostgreSQL, Vercel, JWT/RBAC) is familiar to the project owner, which reduces delivery risk.

5.2 Comparison table

Concern	Selected `[P]`	Alternatives	Trade-off / why not
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS, Radix UI primitives (accessibility)	Remix, SvelteKit, Vite SPA	Next.js gives SSR/RSC for fast first paint on mobile and one deployable; RSC/caching pitfalls are managed by RULES.md
Backend	Next.js route handlers as a modular monolith	NestJS on an always-on host; Hono on Workers	Always-on Node hosting has no verified free option without sleep `[V Render]`; Workers has 10 ms CPU on free `[V]`
Database	PostgreSQL (Supabase)	D1, Firestore, MySQL	See §4.6
ORM/query	Drizzle ORM (schema in TS, reviewed SQL migrations) + raw SQL for scoring/leaderboard	Prisma, Kysely, raw SQL only	Drizzle keeps SQL visible; the owner has raw-SQL experience; Prisma adds engine/connection overhead
Validation	Zod schemas shared client/server	Valibot, Yup	Ecosystem and Drizzle/Zod integration
Auth	Supabase Auth (email/password, verification, reset, TOTP) behind an `AuthProvider` interface; app-owned `users` table	Auth.js, custom auth, Firebase Auth `[U]`, Clerk `[U]`	MAU 50,000 free `[V]`; custom SMTP allowed on Free `[V]`; lock-in reduced by app-owned identity table. Custom auth increases security burden
Realtime	Cached snapshot polling with ETag (baseline) + optional Supabase Broadcast version-ping	Raw WebSockets/SSE on own server; Ably; Firebase RTDB	WebSockets/SSE on serverless are bounded by function duration `[V 300 s]`; managed realtime free caps < 500 `[V]`
Rate limiting	Upstash `@upstash/ratelimit`	In-memory (per instance, ineffective), Postgres counters (hot-path cost)	Serverless has no shared memory; Redis is the standard fit
Jobs	Postgres `jobs` table + tick endpoints (Vercel Cron `[U]`, `pg_cron`+`pg_net` `[U]`, opportunistic ticks)	BullMQ (needs Redis worker), Supabase Edge Functions	No always-on worker available on free hosting
File storage	Supabase Storage (images, exports) behind a `Storage` interface	Cloudflare R2 `[U]`, S3	1 GB free `[V]`; interface allows swap
Email	SMTP via Brevo/Resend	SES `[U]`	Daily caps `[V]` drive queueing design
Testing	Vitest, Testing Library, Playwright, k6	Jest, Cypress	Speed, TS support, k6 for protocol-level load
Observability	Structured JSON logs, Sentry `[U]`, Supabase reports `[V]`, uptime pings	Datadog etc.	Cost

5.3 Vendor lock-in and exit paths

Component	Lock-in	Mitigation
Supabase Postgres	Low (standard Postgres; `pg_dump`)	Only standard extensions in schema; migrations in repo
Supabase Auth	Medium (`auth.users`, JWT)	App `users` table keyed by `auth_user_id`; `AuthProvider` interface; password hashes exportable only per provider terms `[U]` — plan a password-reset migration
Supabase Realtime	Low (optional enhancement)	Polling is the baseline
Supabase Storage	Low	Storage interface; object URLs abstracted
Vercel	Low–medium	Standard Node/Next.js; avoid Vercel-only APIs in domain code; Cron/Firewall usage isolated
Upstash	Low	`RateLimiter` interface

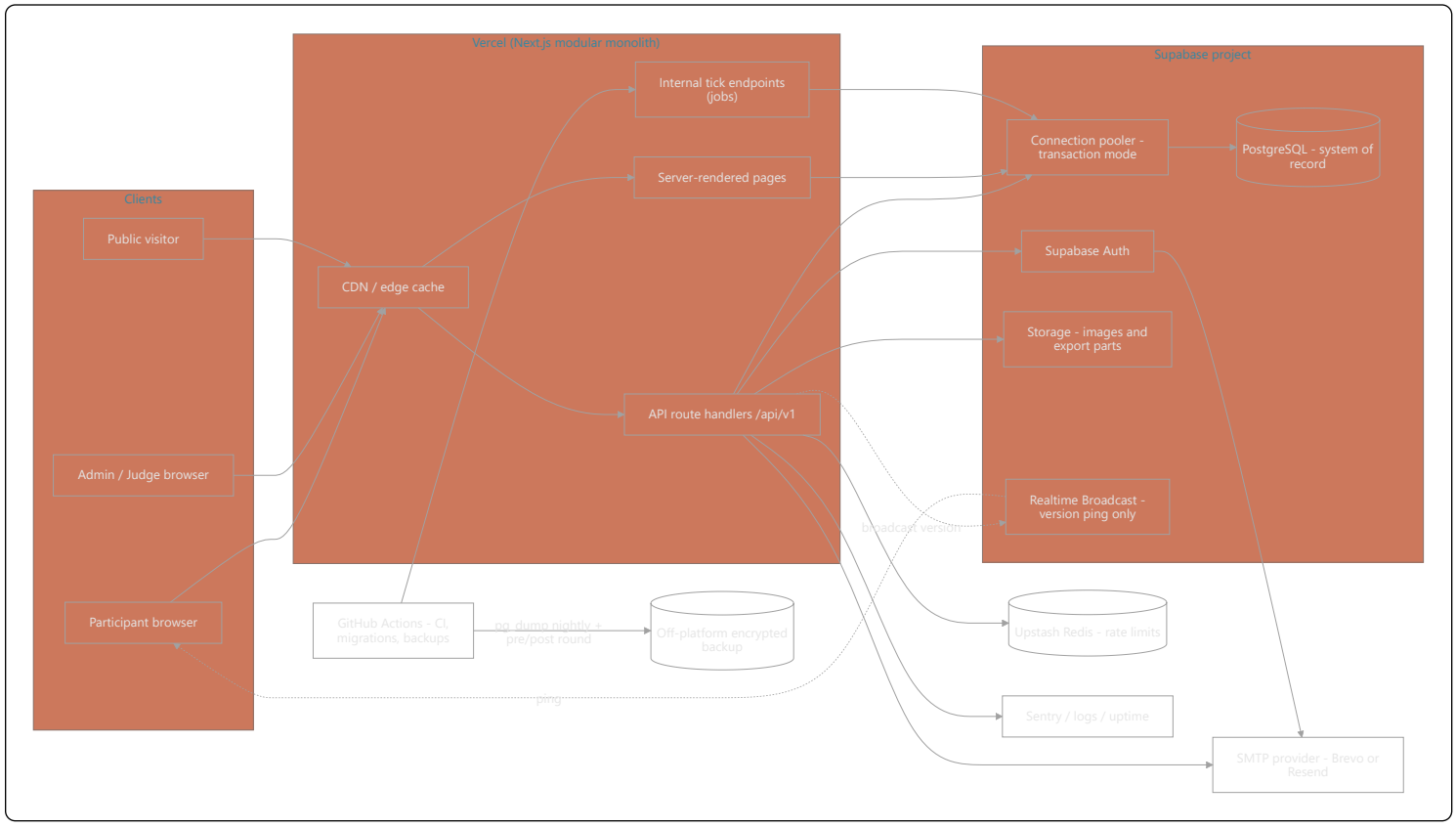
5.4 Migration considerations

Schema changes follow expand → migrate → contract (RULES.md §7). Moving to another Postgres (e.g., Neon) requires a connection-string change plus re-pointing Auth/Storage/Realtime adapters. Moving to Cloudflare would require rewriting the data layer for SQLite semantics and is **not** planned.

---

6. System architecture

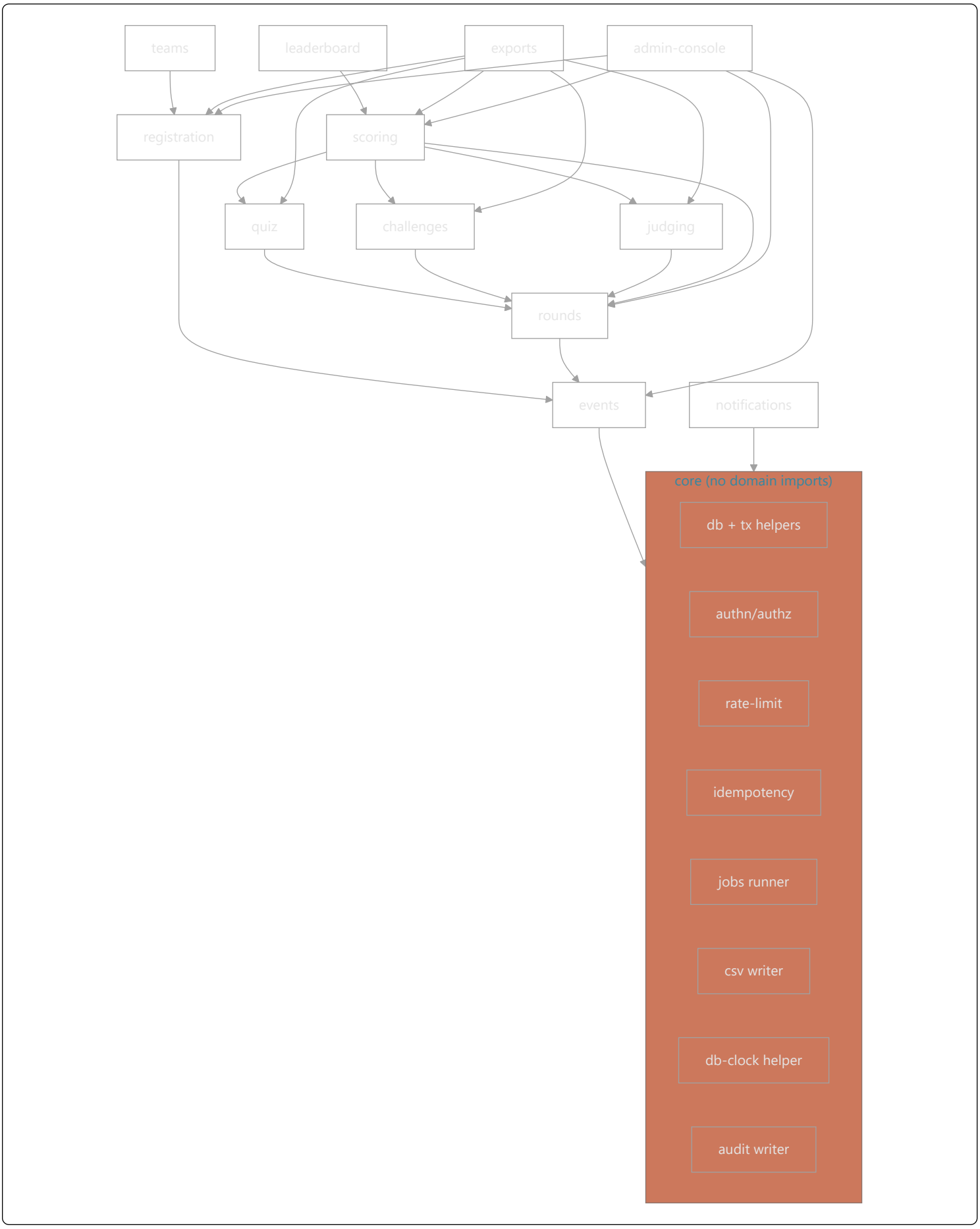
6.1 Runtime architecture



Key properties:

- **Stateless compute.** No authoritative state in function memory (BR-02/03). Any instance can serve any request.
- **One system of record.** All business state in Postgres. Redis holds only rate-limit counters (loss is harmless). Realtime carries only a version number.
- **DB access only from the server.** Browsers never query tables directly. Tables live in a schema not exposed by Supabase's auto-generated API, roles have minimal grants, and RLS is enabled as defense in depth (§7.5).

6.2 Module boundaries



Rules (enforced by lint/dependency-cruiser, RULES.md §2): a module may import another module **only through its `index.ts` public API**; arrows above are the only allowed directions; `core` imports nothing from domain modules; UI components never import repositories.

6.3 Proposed repository structure

```
/
├─ docs/                      # the six planning documents
├─ src/
│  ├─ app/                    # Next.js routes only (thin)
│  │  ├─ (public)/            # landing, events, event detail, public leaderboard
│  │  ├─ (auth)/              # sign-in, sign-up, verify, reset
│  │  ├─ (participant)/e/[slug]/   # dashboard, rounds, quiz, challenge, results
│  │  ├─ (admin)/admin/…      # admin console
│  │  ├─ (judge)/judge/…      # judging workspace
│  │  └─ api/v1/…             # route handlers (call module services)
│  ├─ modules/
│  │  ├─ events/  registration/  teams/  rounds/  quiz/  challenges/
│  │  ├─ judging/  scoring/  leaderboard/  exports/  notifications/  admin-console/
│  │  │   └─ each: index.ts (public API) · domain/ · service/ · repo/ · schemas/ · ui/ · __tests__/
│  ├─ round-types/            # plugin registry: quiz/, challenge/, judged/  (§10)
│  ├─ core/                   # db, tx, auth, rbac, errors, logger, rate-limit, idempotency, jobs, csv, clock, audit, config
│  ├─ components/ui/          # design-system primitives (DESIGN.md)
│  └─ lib/                    # pure helpers (formatting, ids)
├─ db/
│  ├─ migrations/             # reviewed SQL migrations (expand/contract)
│  ├─ policies/               # RLS policies and grants as SQL
│  └─ seeds/                  # deterministic dev/test seeds
├─ tests/
│  ├─ integration/  e2e/  security/  recovery/
│  └─ load/                   # k6 scripts + scenario configs
├─ scripts/                   # backup, restore drill, reconcile, seed-load
├─ .github/workflows/         # ci.yml, migrate.yml, backup.yml, loadtest.yml
├─ .env.example
└─ package.json
```

A monorepo/workspaces split is intentionally **not** used in the MVP.

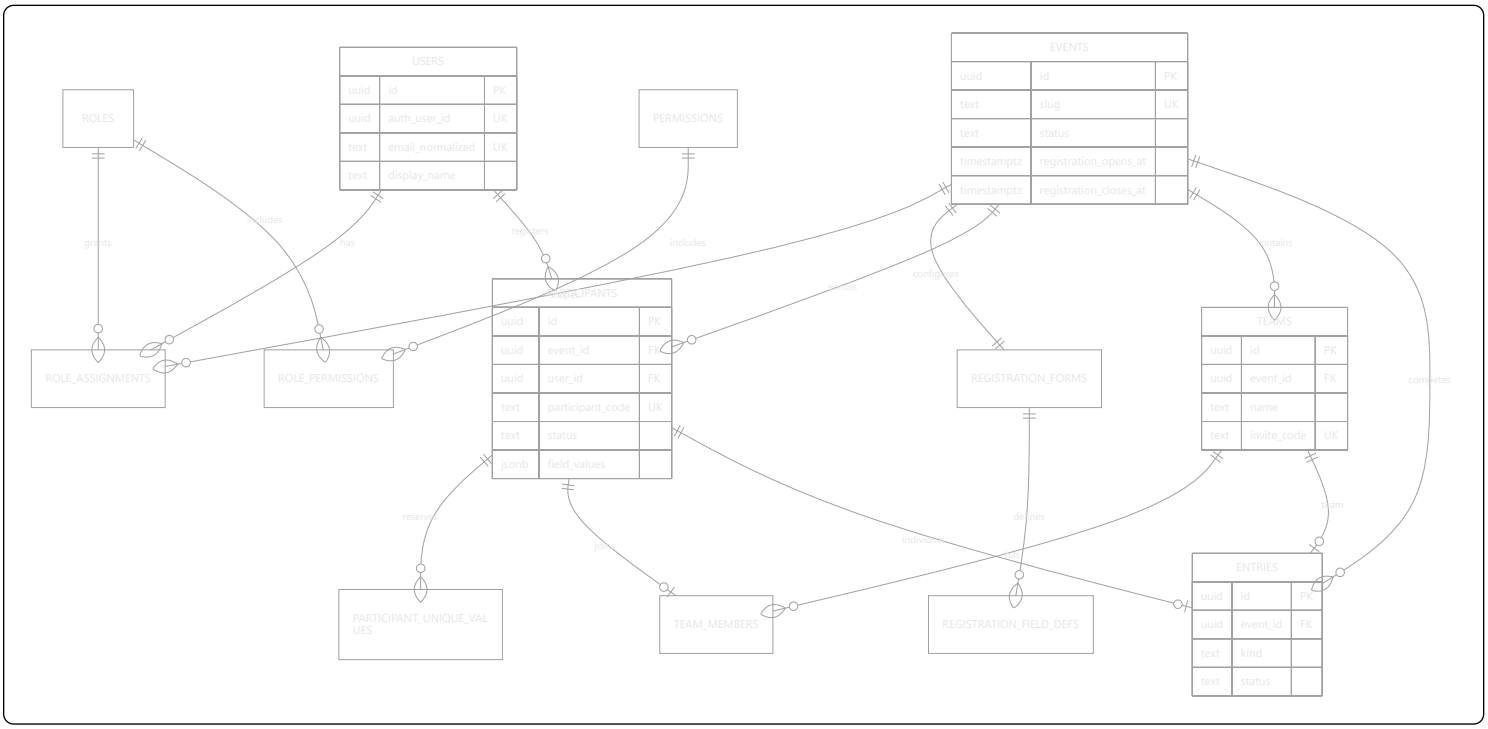
7. Data model

7.1 Conventions

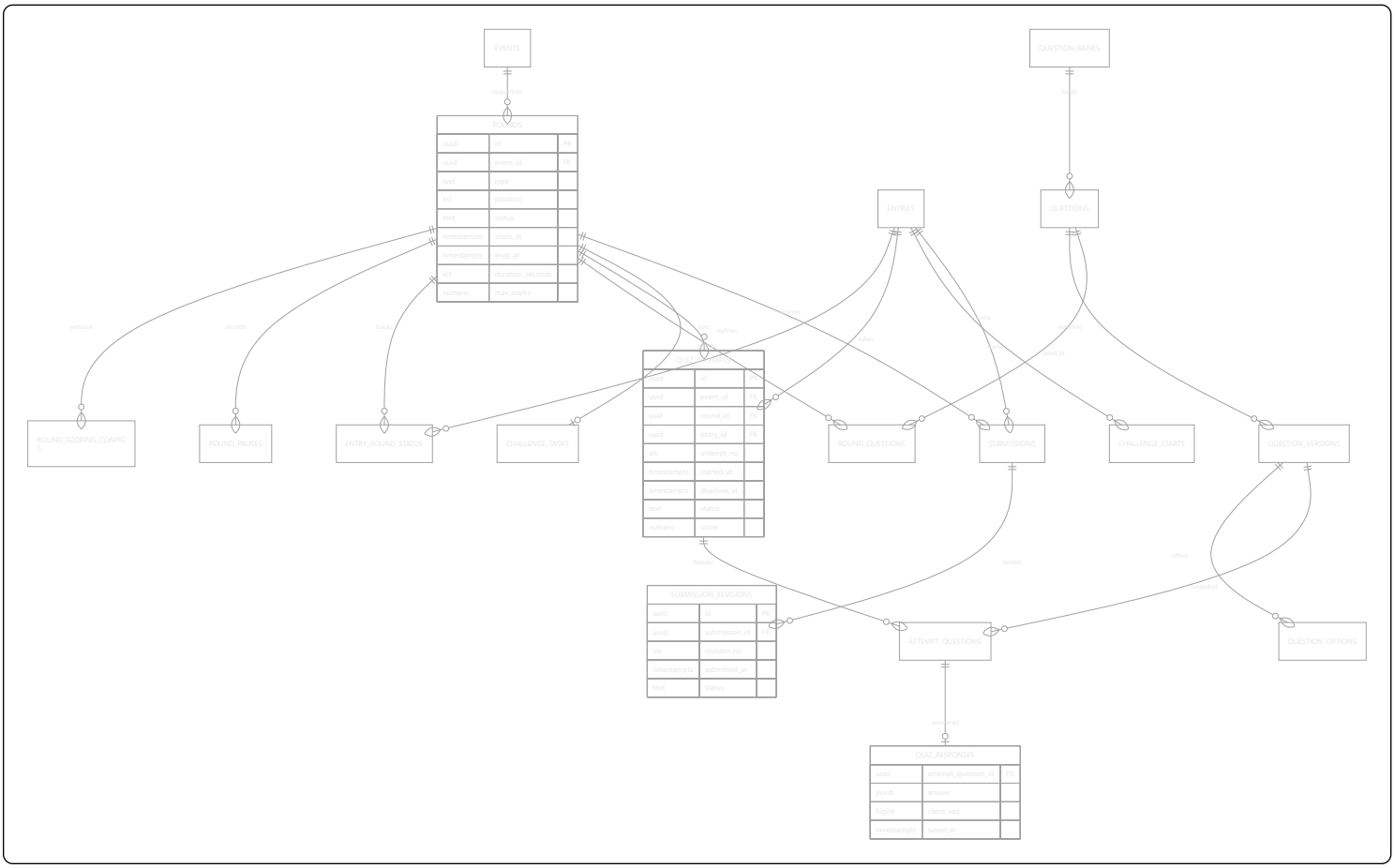
- Primary keys: `uuid` (v7 where available for index locality; otherwise `gen_random_uuid()`), except counters/sequences noted.
- All timestamps `timestamptz`, UTC. Authoritative time = `now()` evaluated by Postgres (BR-02).
- Every event-scoped table has `event_id uuid NOT NULL`. Parent tables expose `UNIQUE (event_id, id)` so children use **composite foreign keys** `(event_id, parent_id)`, making cross-event references impossible at the database level (BR-01).
- Soft-delete only where history matters (`deleted_at`); scoring/audit tables are append-only.
- Enums as Postgres enums or `text CHECK` (chosen at M1; changes need expand/contract migrations).
- `jsonb` only for organizer-configurable shapes (field values, round config, scoring config) validated by Zod at write time and versioned.

7.2 Entity relationship diagrams

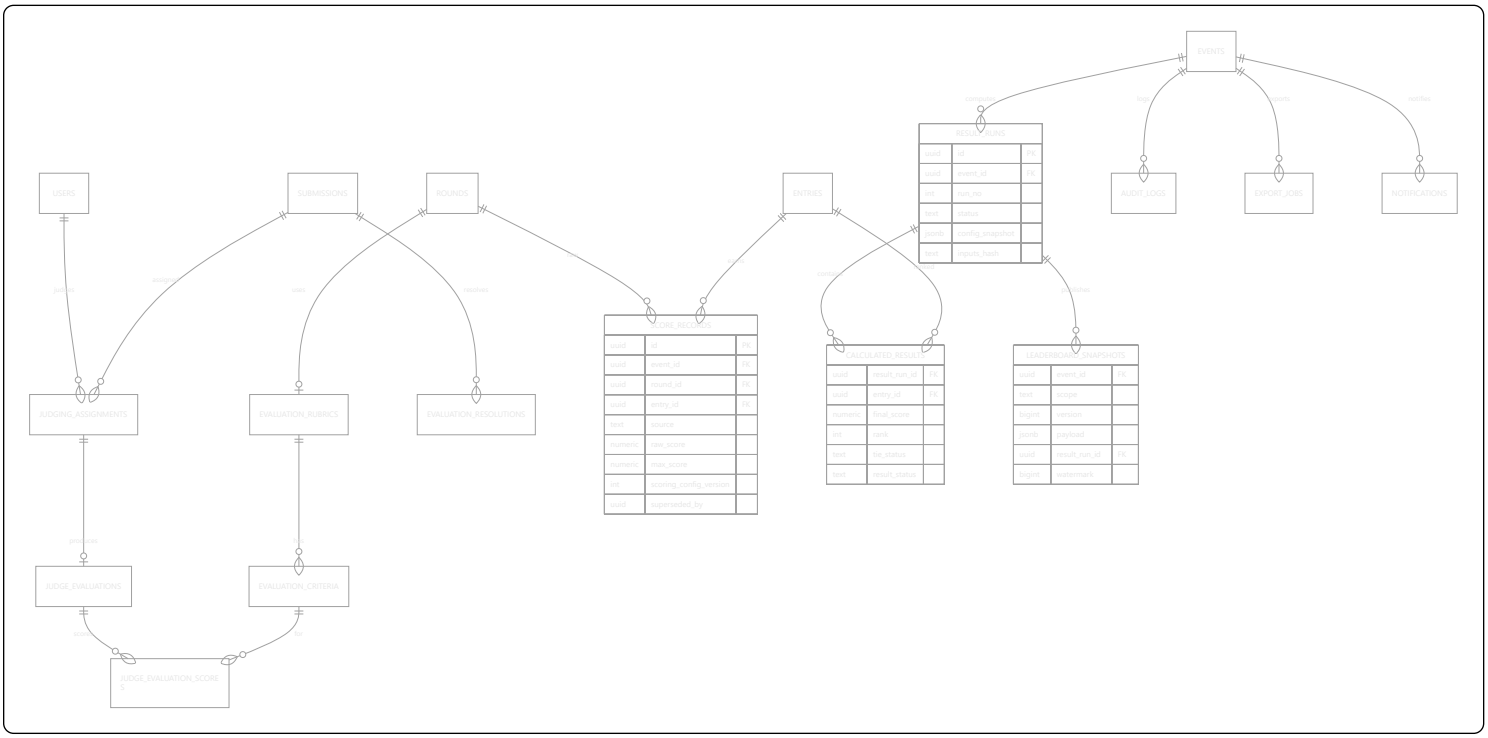
Identity, events, registration and teams



Rounds, quiz and challenge



Judging, scoring, leaderboard, operations



7.3 Table catalogue

`PK` primary key, `UQ` unique, `FK` foreign key (composite `(event_id, x)` where noted). Index lists show the intended access paths, not every trivial index.

Identity and access

Table	Purpose	Key columns	Constraints	Ind
`users`	App identity mapped to Supabase Auth	`id`, `auth_user_id`, `email_normalized`, `display_name`, `email_verified_at`, `disabled_at`	UQ(`auth_user_id`), UQ(`email_normalized`)	—
`roles`	Fixed role catalogue	`key` (`super_admin`, `event_admin`, `round_manager`, `judge`, `reviewer`)	PK(`key`)	—
`permissions`	Permission keys (§8.2)	`key`	PK	—
`role_permissions`	Role → permission	`role_key`, `permission_key`	PK(both)	—
`role_assignments`	Scoped grants	`id`, `user_id`, `role_key`, `scope_type` (`global`/`event`/`round`), `event_id`, `round_id`, `granted_by`, `revoked_at`	CHECK scope columns consistent; UQ(`user_id`, `role_key`, `scope_type`, `event_id`, `round_id`) where not revoked	(`u` (`e

Events, registration, teams

Table	Purpose	Key columns	Constraints	Indexes
`events`	An independent competition	`id`, `slug`, `title`, `description_md`, `banner_asset_id`, `status`, `registration_opens_at`, `registration_closes_at`, `starts_at`, `ends_at`, `timezone`, `participant_limit`, `mode` (`individual`/`team`/`both`), `team_min`, `team_max`, `team_lock_at`, `require_email_verification`, `approval_mode`, `leaderboard_policy jsonb`, `result_visibility`, `result_config jsonb`, `result_config_version`	UQ(`slug`); UQ(`id`) ; CHECK windows ordered	(`status`,`starts_at`)
`registration_forms`	Versioned form per event	`id`, `event_id`, `version`, `is_active`	UQ(`event_id`,`version`); partial UQ active per event	—
`registration_field_defs`	Field definitions	`id`, `form_id`, `key`, `label`, `type`, `required`, `validation jsonb`, `options jsonb`, `is_unique`, `is_pii`, `position`	UQ(`form_id`,`key`)	(`form_id`,`position`)
`participants`	A user's registration in an event (participant record)	`id`, `event_id`, `user_id`, `participant_code`, `status`, `form_id`, `field_values jsonb`, `registered_at`, `verified_at`, `status_reason`	UQ(`event_id`,`user_id`); UQ(`event_id`,`participant_code`); UQ(`event_id`,`id`)	(`event_id`,`status`), GIN(`field_values`) optional, trigram on searchable name/email `[E]`
`participant_unique_values`	Enforces organizer-defined unique fields	`event_id`, `field_key`, `normalized_value`, `participant_id`	UQ(`event_id`,`field_key`,`normalized_value`)	—
`event_counters`	Race-free participant/team numbering	`event_id`, `counter_key`, `value`	PK(`event_id`,`counter_key`); incremented via `UPDATE … RETURNING`	—
`teams`	Team in an event	`id`, `event_id`, `name`, `name_normalized`, `captain_participant_id`, `invite_code`, `status`, `locked_at`	UQ(`event_id`,`name_normalized`); UQ(`invite_code`); UQ(`event_id`,`id`)	(`event_id`,`status`)
`team_members`	Roster	`team_id`, `event_id`, `participant_id`, `role`, `joined_at`, `left_at`	FK(`event_id`,`team_id`), FK(`event_id`,`participant_id`); partial UQ(`event_id`,`participant_id`) where `left_at` is null (one active team per event)	(`team_id`)
`entries`	Unified competing unit	`id`, `event_id`, `kind` (`individual`/`team`), `participant_id` | `team_id`, `public_code`, `status` (`active`,`withdrawn`,`disqualified`), `status_reason`	CHECK exactly one of participant/team; UQ(`event_id`,`participant_id`); UQ(`event_id`,`team_id`); UQ(`event_id`,`id`)	(`event_id`,`status`)

Rounds

Table	Purpose	Key columns	Constraints	Indexes
`rounds`	Round definition	`id`, `event_id`, `type`, `title`, `instructions_md`, `position`, `status`, `starts_at`, `ends_at`, `duration_seconds`, `max_marks`, `type_config jsonb`, `type_config_version`, `eligibility jsonb`, `advancement jsonb`, `visibility jsonb`, `late_policy jsonb`, `active_scoring_config_id`, `paused_at`	UQ(`event_id`,`id`); UQ(`event_id`,`position`) deferrable	(`event_id`,`position`), (`event_id`,`status`)
`round_scoring_configs`	**Immutable, versioned** scoring config	`id`, `round_id`, `event_id`, `version`, `config jsonb`, `created_by`, `created_at`, `superseded_at`	UQ(`round_id`,`version`); update trigger forbids change once referenced	—
`round_pauses`	Pause intervals (BR-14)	`id`, `round_id`, `paused_at`, `resumed_at`, `extended_seconds`, `by_user`	—	(`round_id`)
`entry_round_status`	Eligibility/advancement per entry per round	`event_id`, `round_id`, `entry_id`, `eligibility`, `advancement`, `override_reason`, `updated_by`	PK(`round_id`,`entry_id`)	(`round_id`,`advancement`)

Quiz

Table	Purpose	Key columns	Constraints	Indexes
`question_banks`	Group of questions	`id`, `event_id`, `name`	UQ(`event_id`,`name`)	—
`questions`	Logical question	`id`, `event_id`, `bank_id`, `tags text[]`, `difficulty`, `current_version_id`, `archived_at`	UQ(`event_id`,`id`)	GIN(`tags`)
`question_versions`	**Immutable** content snapshot	`id`, `question_id`, `event_id`, `version_no`, `type` (`single`,`multi`,`truefalse`,`text`), `stem_md`, `stem_asset_id`, `marks`, `negative_marks`, `partial_credit`, `text_config jsonb`, `explanation_md`	UQ(`question_id`,`version_no`); trigger forbids update	—
`question_options`	Options of a version	`id`, `version_id`, `label`, `text_md`, `asset_id`, `is_correct`, `position`	UQ(`version_id`,`position`); CHECK ≥1 correct for choice types (enforced in service + deferred trigger)	(`version_id`)
`round_questions`	Quiz composition	`round_id`, `question_id`, `position`, `marks_override`, `pool_key`	PK(`round_id`,`question_id`)	(`round_id`,`position`)
`quiz_attempts`	One attempt	`id`, `event_id`, `round_id`, `entry_id`, `participant_id`, `attempt_no`, `status`, `started_at`, `deadline_at`, `submitted_at`, `submit_source` (`manual`/`auto`/`admin`), `seed`, `score`, `max_score`, `idempotency_key`	UQ(`round_id`,`entry_id`,`participant_id`,`attempt_no`); UQ(`idempotency_key`); UQ(`event_id`,`id`)	(`round_id`,`status`), (`round_id`,`deadline_at`) `status='in_progress'` (sw
`attempt_questions`	Frozen ordered questions for an attempt	`id`, `attempt_id`, `event_id`, `question_version_id`, `position`, `option_order int[]`	UQ(`attempt_id`,`position`)	(`attempt_id`)
`quiz_responses`	Saved/submitted answer	`attempt_question_id`, `event_id`, `answer jsonb`, `client_seq`, `saved_at`, `awarded_marks`, `is_correct`, `needs_manual_review`	PK(`attempt_question_id`); upsert guarded by `client_seq`	(`event_id`,`attempt_quest for export keyset

Development challenge

Table	Purpose	Key columns	Constraints	Indexes
`challenge_tasks`	Task definition for a round	`id`, `event_id`, `round_id`, `description_md`, `instructions_md`, `resources jsonb`, `permitted_tools jsonb`, `deliverables jsonb`, `field_defs jsonb`, `clock_mode` (`fixed_window`/`per_entry_clock`), `time_limit_seconds`	UQ(`round_id`)	—
`challenge_starts`	Per-entry clock start	`round_id`, `entry_id`, `event_id`, `started_at`, `deadline_at`	PK(`round_id`,`entry_id`)	—
`submissions`	Logical submission per entry per round	`id`, `event_id`, `round_id`, `entry_id`, `counted_revision_id`, `status`	UQ(`round_id`,`entry_id`); UQ(`event_id`,`id`)	(`round_id`,`status`)
`submission_revisions`	Every submit	`id`, `submission_id`, `event_id`, `revision_no`, `repo_url`, `deploy_url`, `video_url`, `docs_url`, `description`, `extra jsonb`, `submitted_at`, `is_late`, `is_draft`, `idempotency_key`	UQ(`submission_id`,`revision_no`); UQ(`idempotency_key`)	(`submission_id`,`revision_no` desc), (`event_id`,`submitted_at`)

Judging

Table	Purpose	Key columns	Constraints	Indexes
`evaluation_rubrics`	Rubric for a round	`id`, `event_id`, `round_id`, `aggregation` (`mean`,`median`,`trimmed_mean`,`max`,`min`,`weighted_judges`), `trim_pct`, `variance_threshold_pct`, `blind`, `min_judges`	UQ(`round_id`)	—
`evaluation_criteria`	Criteria	`id`, `rubric_id`, `name`, `description`, `max_marks`, `weight`, `position`	UQ(`rubric_id`,`position`); CHECK `max_marks > 0`	—
`judging_assignments`	Item → judge	`id`, `event_id`, `round_id`, `submission_id` | `entry_id`, `judge_user_id`, `status`, `judge_weight`, `coi_declared`	UQ(`round_id`,`entry_id`,`judge_user_id`)	(`judge_user`, (`round_id`,
`judge_evaluations`	A judge's evaluation	`id`, `assignment_id`, `event_id`, `status` (`draft`,`submitted`,`locked`), `total`, `comment`, `evidence_url`, `submitted_at`, `revision_no`	UQ(`assignment_id`,`revision_no`)	—
`judge_evaluation_scores`	Per-criterion marks	`evaluation_id`, `criterion_id`, `marks`, `comment`	PK(both); CHECK `0 ≤ marks` and ≤ criterion max (service + trigger)	—
`evaluation_resolutions`	Disagreement resolution	`id`, `event_id`, `round_id`, `entry_id`, `method`, `resolved_score`, `reason`, `resolved_by`, `resolved_at`	—	(`round_id`,

Scoring, results, leaderboard

Table	Purpose	Key columns	Constraints	Inde
`score_records`	**Append-only** raw scores	`id`, `event_id`, `round_id` (nullable = event-level adjustment/penalty), `entry_id`, `source` (`quiz_auto`, `quiz_manual`, `judge_aggregate`, `admin_adjust`, `penalty`), `raw_score`, `max_score`, `scoring_config_version`, `source_ref`, `reason`, `created_by`, `created_at`, `superseded_by`, `superseded_at`	Update/delete forbidden by trigger (only `superseded_by`/`superseded_at` set once); partial UQ(`round_id`, `entry_id`) where `source in (quiz_auto,quiz_manual,judge_aggregate)` and `superseded_by is null` (one primary record)	(`ev` whe (`ev` chec
`result_runs`	Versioned calculation	`id`, `event_id`, `run_no`, `status` (`provisional`, `final`, `superseded`), `config_snapshot jsonb`, `inputs_hash`, `input_watermark`, `created_by`, `created_at`, `published_at`	UQ(`event_id`, `run_no`)	(`ev`
`calculated_results`	Per-entry cumulative result	`result_run_id`, `event_id`, `entry_id`, `round_scores jsonb`, `weighted_total`, `penalties`, `final_score`, `rank`, `tie_status`, `result_status`, `advancement jsonb`	PK(`result_run_id`, `entry_id`)	(`re`
`leaderboard_aggregates`	Rank rows used to build snapshots	`event_id`, `scope`, `entry_id`, `rank`, `score`, `result_run_id`	PK(`event_id`, `scope`, `entry_id`)	(`ev`
`leaderboard_snapshots`	Serving payloads	`event_id`, `scope`, `version`, `source` (`live`/`run`), `payload jsonb` (top-N), `result_run_id` (null when `live`), `watermark` (max `score_records.id` included), `generated_at`, `etag`	PK(`event_id`, `scope`, `version`); latest via index	(`ev` desc

Operations

Table	Purpose	Key columns	Notes
`audit_logs`	Append-only audit	`id`, `event_id`, `actor_user_id`, `actor_role`, `action`, `target_type`, `target_id`, `before jsonb`, `after jsonb`, `reason`, `ip_hash`, `request_id`, `created_at`	UPDATE/DELETE revoked and blocked by trigger (BR-29); index (`event_id`, `created_at` desc), (`target_type`, `target_id`)
`idempotency_keys`	Replay protection for non-natural-key writes	`key`, `scope`, `request_hash`, `response jsonb`, `created_at`, `expires_at`	Natural-key writes (attempt start/submit, submission) use unique constraints instead
`jobs`	DB-backed job queue	`id`, `type`, `payload`, `run_at`, `attempts`, `locked_by`, `locked_until`, `status`, `last_error`	Claimed with `FOR UPDATE SKIP LOCKED`
`export_jobs`	Export lifecycle	`id`, `event_id`, `type`, `filters`, `fields`, `status`, `cursor`, `rows_written`, `parts int`, `requested_by`, `expires_at`, `checksum`	See §16
`notifications`	In-app and email outbox	`id`, `event_id`, `user_id`, `channel`, `template`, `payload`, `status`, `send_after`, `attempts`	Rate-limited sender honoring provider caps
`media_assets`	Uploaded images	`id`, `event_id`, `storage_path`, `mime`, `bytes`, `sha256`, `uploaded_by`	Size/mime allow-list

7.4 Supporting concurrency, scoring and export workloads

Need	Mechanism
Race-free capacity and numbering	Registration runs in one transaction: `SELECT … FROM events WHERE id=$1 FOR UPDATE` (short) or advisory lock keyed by event, capacity check, `event_counters` increment, insert
Duplicate submissions	Natural unique constraints + idempotency keys (`quiz_attempts.idempotency_key`, `submission_revisions.idempotency_key`)
Concurrent answer saves	`INSERT … ON CONFLICT (attempt_question_id) DO UPDATE … WHERE excluded.client_seq > quiz_responses.client_seq` (last-sequence wins, never regress)
Exactly-once submit	`SELECT … FOR UPDATE` on the attempt row; status transition guarded (`in_progress` → `submitted`)
Efficient result calculation	Round scores are single-row-per-source in `score_records` with partial index on non-superseded rows; a result run is one set-based SQL/TS pass over ≈ 500 entries × R rounds (`[E]` well under a second)
Large CSV exports	Keyset pagination by primary key, covering indexes on `(event_id, …, id)`, a read-only role with `statement_timeout`, chunked resumable jobs (§16)
Hot-path isolation	Leaderboard reads hit a single precomputed `leaderboard_snapshots` row, never aggregate tables

7.5 Data isolation strategy

1. **Structural:** `event_id` on every event-scoped table plus composite FKs (`(event_id, parent_id)`), so a child can never reference another event's parent.
2. **Application:** every repository function takes an explicit `EventScope` argument; there is no repository method that queries event data without it; a unit test scans repositories for unscoped queries.
3. **Authorization:** role assignments are scoped (`global`/`event`/`round`); judges see only assigned items (BR-19).
4. **Database policy (defense in depth):** app tables live in a non-exposed schema; `anon`/`authenticated` roles have no grants; RLS is enabled with deny-by-default policies so an accidental exposure yields no rows. The server connects with a least-privilege role (`app_rw`); exports use `app_ro`; migrations use a separate owner role. `[U]` exact Supabase role/schema configuration is validated in T-1.3.
5. **Testing:** automated cross-event leakage tests (SEC-ISO-*) attempt to read/write event B data with event A credentials on every endpoint.

8. Authentication and authorization

8.1 Authentication

- **Provider:** Supabase Auth (email + password), behind an `AuthProvider` interface. Users are mirrored into `users` (app identity) on first sign-in.
- **Sessions:** HTTP-only, `Secure`, `SameSite=Lax` cookies managed via the Supabase SSR helper. Access-token verification on each request should use local JWT verification against the project's signing keys `[U — confirm asymmetric signing/JWKS support]`, falling back to `auth.getUser()` (adds a network call) on hot paths only if necessary.
- **Email verification:** per-event flag `require_email_verification` (FR-AUTH-02). Verification emails go through custom SMTP; per-hour Auth rate limits are raised deliberately `[S]`.
- **MFA:** TOTP for `super_admin` and `event_admin` (Supabase Free includes basic MFA `[V]`) `[P]` D-P10.
- **Brute-force protection:** IP+account rate limits (Upstash) and Supabase Auth's built-in limits; optional CAPTCHA `[P]`.
- **Session revocation:** role removal and password change trigger refresh-token revocation.

8.2 Permission catalogue and role matrix

◉ allowed within assigned scope · ○ read-only variant · blank = denied. Participants act only on **their own** records; the public is unauthenticated.

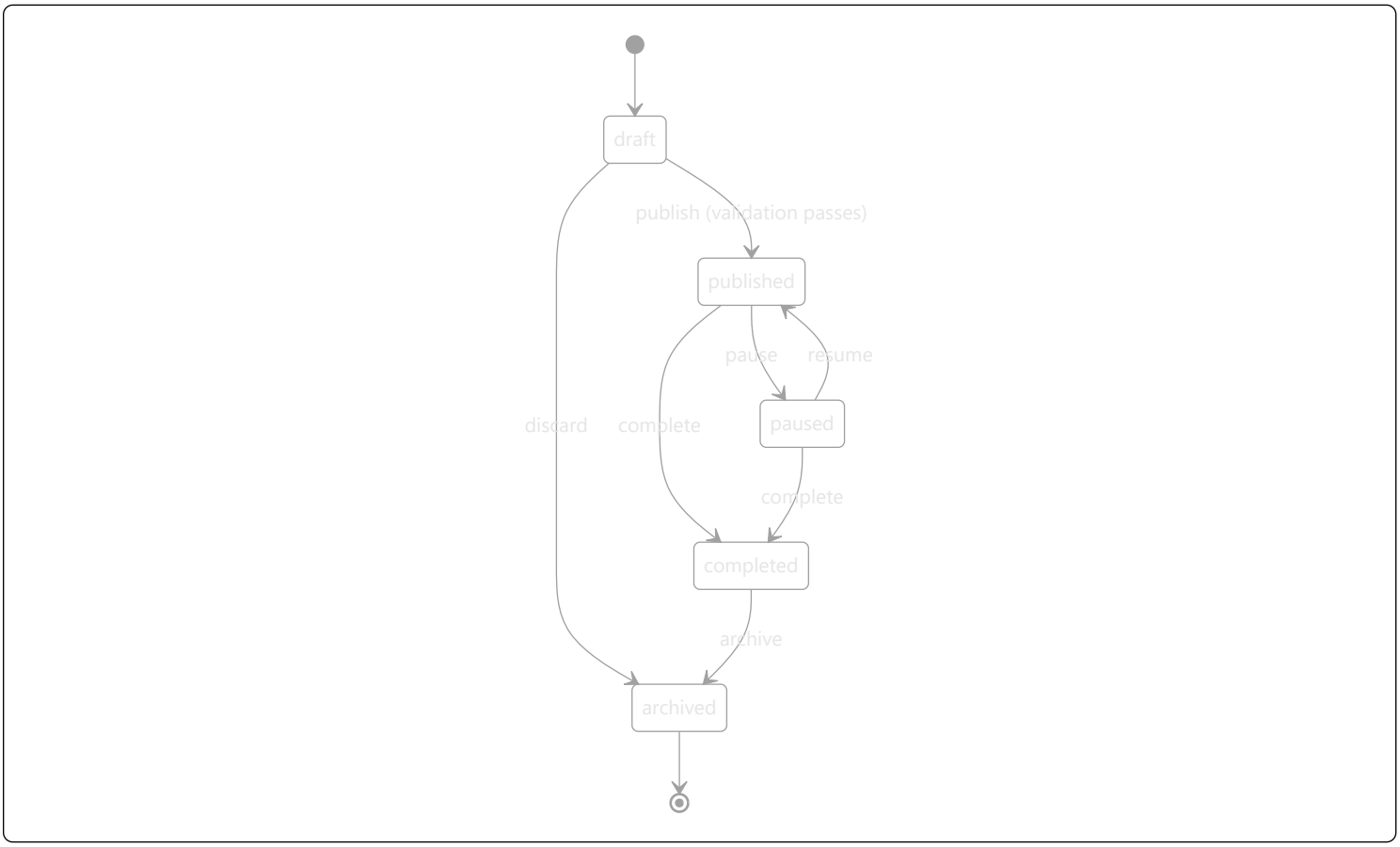
Permission	Super admin	Event admin	Round manager	Judge	Reviewer	Participant
`event:create`	•					
`event:update` / `event:lifecycle` (publish, pause, resume, complete, archive)	•	•				
`event:assign_roles`	•	•				
`registration:read`	•	•	○		○	own
`registration:review` (approve/reject/edit status)	•	•				
`team:manage` (override)	•	•				captain of own
`round:manage` (create/edit/reorder)	•	•	• (assigned)			
`round:control` (open, close, pause, extend)	•	•	• (assigned)			
`quiz:author` / `challenge:author` / `rubric:manage`	•	•	• (assigned)			
`quiz:preview`	•	•	•		○	
`attempt:take` / `submission:create`						• own, when eligible
`submission:read`	•	•	•	assigned	○	own
`judging:assign`	•	•	•			
`judging:score`				• assigned		
`judging:reconcile`	•	•	•			
`scores:finalize` / `results:publish`	•	•				
`scores:unlock`	•	• (with reason)				
`leaderboard:configure`	•	•				
`leaderboard:view`	•	•	•	• (if policy)	•	per policy
`export:create`	•	•	○ non-PII		○ non-PII	own data only
`export:pii`	•	•				
`audit:read`	•	• (own events)			○	
`system:settings`	•					

8.3 Enforcement

- A single `authorize(actor, permission, scope)` function is the only allow/deny decision point. Every route is declared with `route({ permission, scope, rateLimit, idempotent })`; a registry test fails the build if a route lacks a policy (deny by default).
- **Resource-level checks** (ABAC) follow permission checks: ownership (`participant.user_id = actor`), assignment (`judging_assignments.judge_user_id = actor`), event scope, event status, round status and time window.
- Sensitive operations write to `audit_logs` in the **same transaction** as the change.

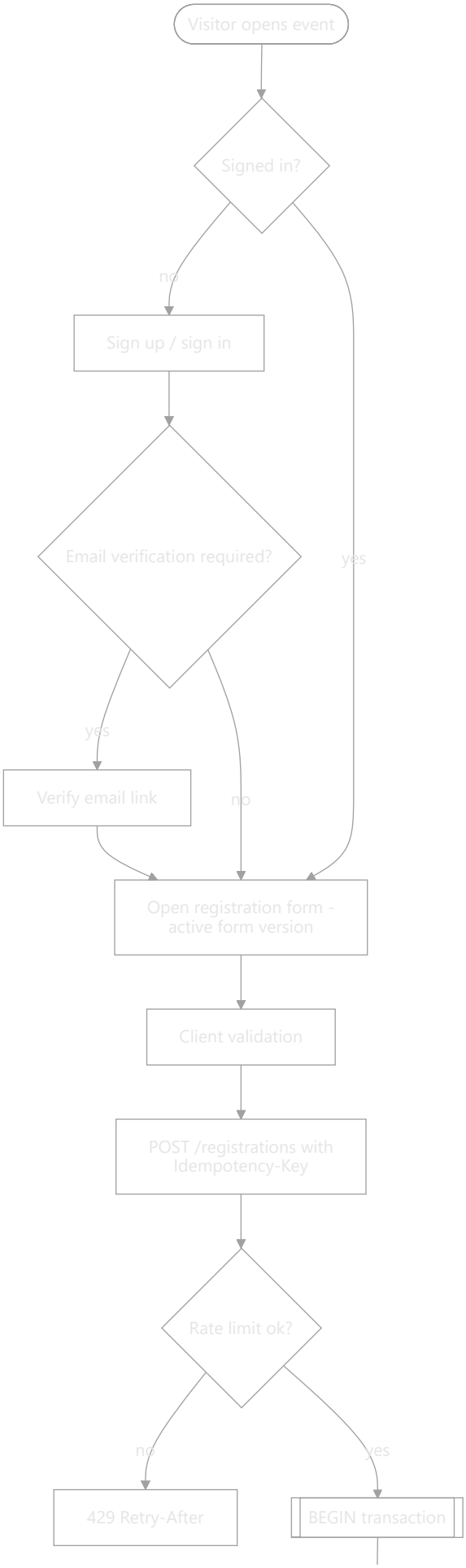
9. Event, registration and round lifecycles

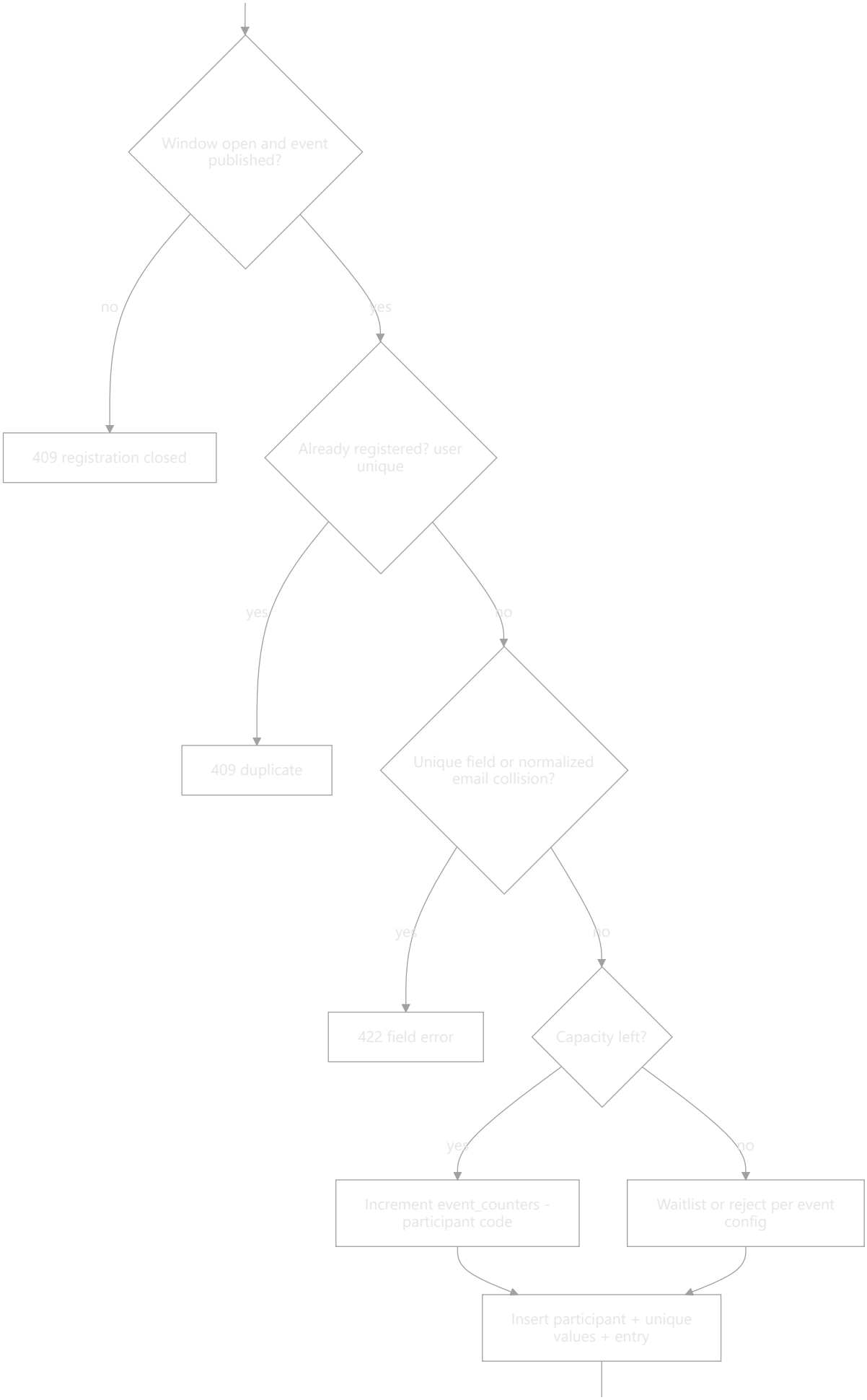
9.1 Event lifecycle

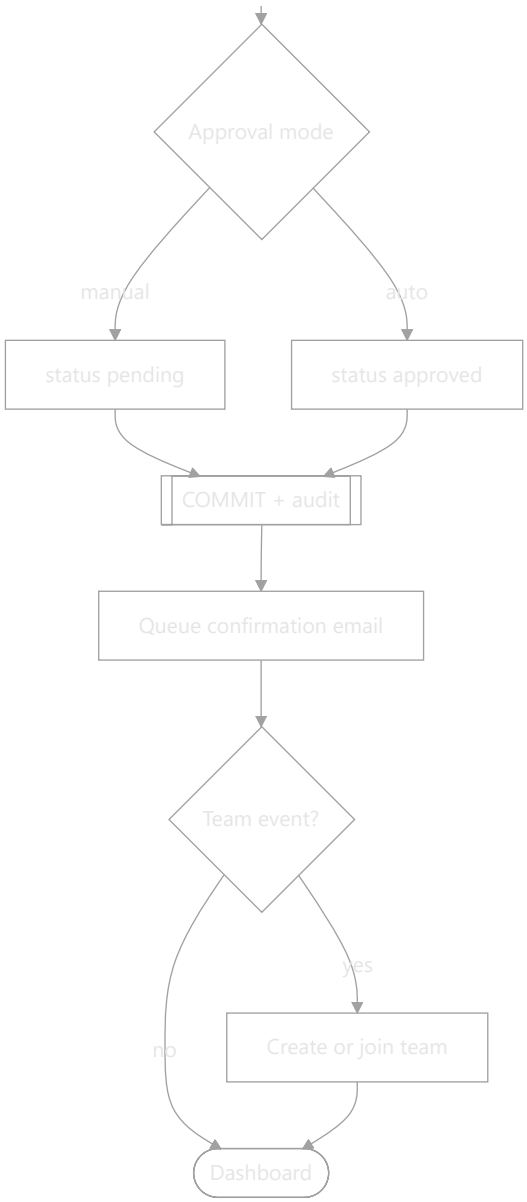


`published` exposes the event publicly. `registration_open` is **derived**: `status in (published)` ∧ window ∧ capacity, with an optional manual override. `completed` freezes participant writes; `archived` makes the event read-only (exports still permitted, FR-EVT-07).

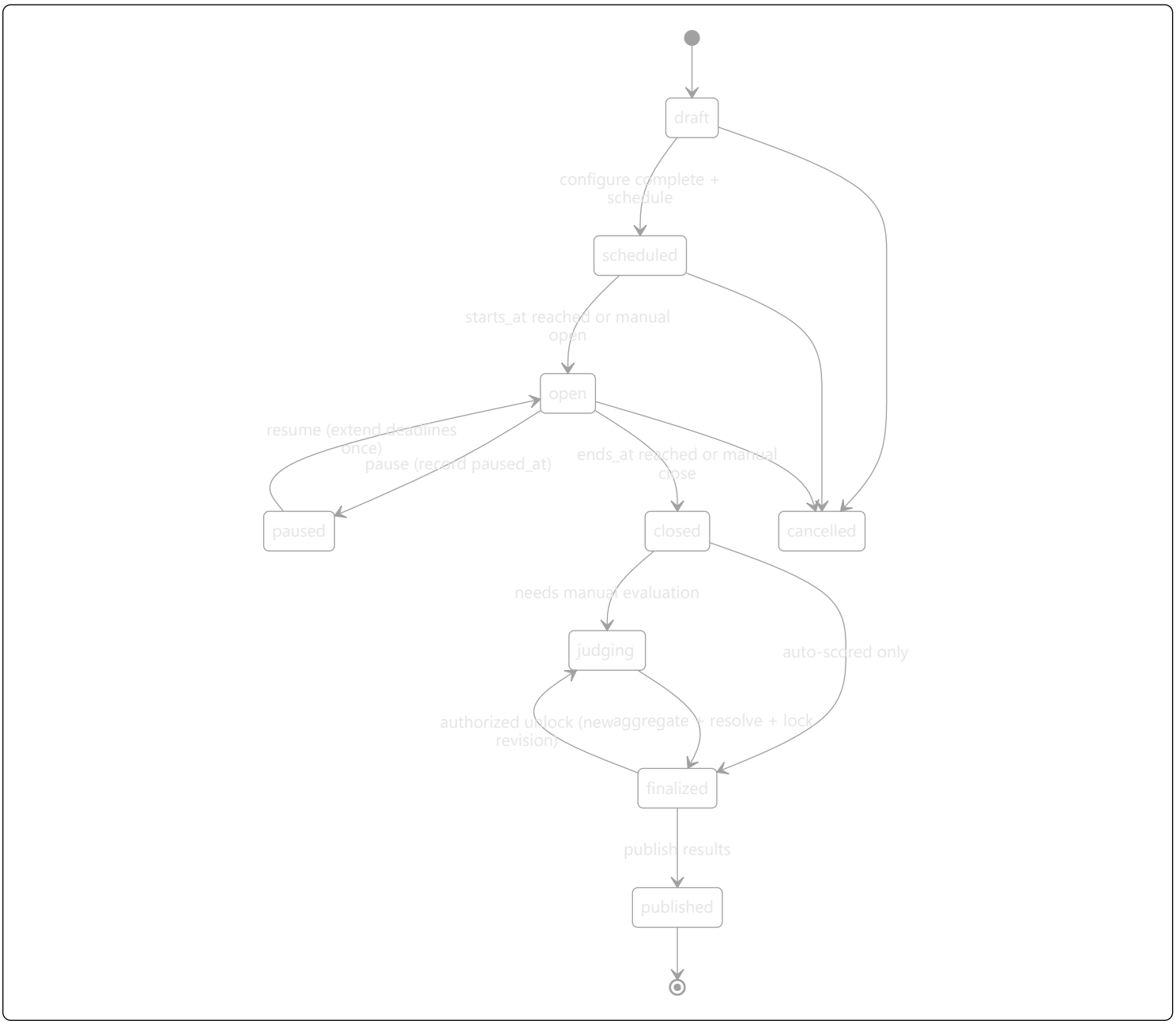
9.2 Registration workflow


Visitor opens event
Signed in?
no
Sign up / sign in
Email verification required?
yes
yes
Verify email link
no
Open registration form - active form version
Client validation
POST /registrations with Idempotency-Key
Rate limit ok?
no
429 Retry-After
BEGIN transaction


Window open and event published?
no
yes
409 registration closed
Already registered? user unique
yes
no
409 duplicate
Unique field or normalized email collision?
yes
no
422 field error
Capacity left?
yes
no
Increment event_counters - participant code
Waitlist or reject per event config
Insert participant + unique values + entry

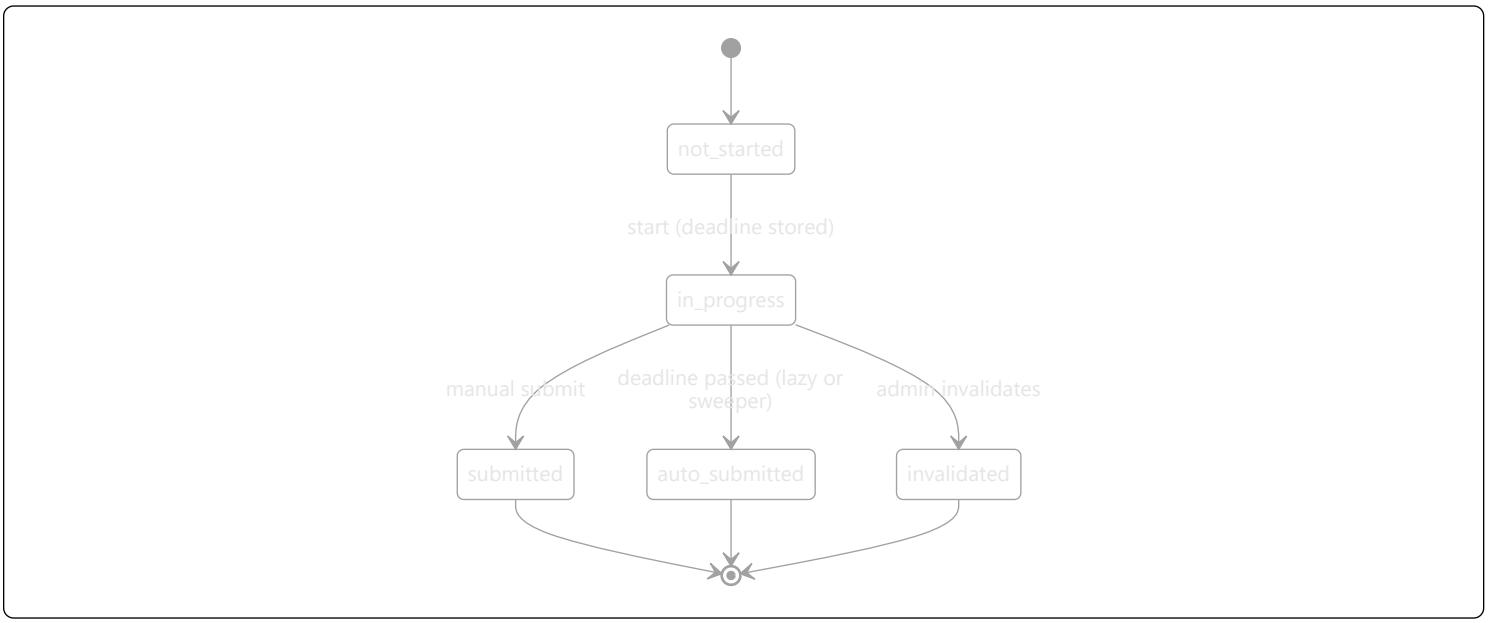

Approval mode
manual
auto
status pending
status approved
COMMIT + audit
Queue confirmation email
Team event?
yes
no
Create or join team
Dashboard

9.3 Round lifecycle



Transitions are performed by explicit server actions guarded by permission, current state and time. `scheduled → open` and `open → closed` are **time-derived** where possible (evaluated against the DB clock on each request and by the sweeper job) so a missed cron tick cannot leave a round open past `ends_at`.

9.4 Attempt lifecycle (quiz)



10. Round engine

10.1 Design

Round count, type and order are **data** (BR-30). Behavior specific to a round type lives in a **round-type plugin** registered in `src/round-types/`. The core engine handles what is common: lifecycle, timing, eligibility, advancement, score record intake, audit, export column registration.

ts

```ts
// src/round-types/contract.ts  (illustrative — not yet implemented)
export interface RoundType<TConfig = unknown> {
  readonly id: string;                        // 'quiz' | 'challenge' | 'judged' | future ids
  readonly configVersion: number;
  readonly configSchema: ZodType<TConfig>;    // validates rounds.type_config
  migrateConfig?(fromVersion: number, raw: unknown): TConfig;

  validateForOpen(ctx: RoundContext): ValidationIssue[];       // blocks opening if invalid
  participantView(ctx: RoundContext, entry: EntryRef): Promise<ParticipantRoundView>;
  handleAction(ctx: RoundContext, entry: EntryRef, action: string,
               input: unknown, tx: Tx): Promise<ActionResult>;  // e.g. start, save, submit
  onClose(ctx: RoundContext, tx: Tx): Promise<void>;            // finalize attempts, etc.
  produceRawScores(ctx: RoundContext, tx: Tx): AsyncIterable<RawScoreInput>;
  exportColumns(ctx: RoundContext): ExportColumnDef[];
  adminPanels: AdminPanelDef[];                                  // UI extension points
}
```

Adding a new round type = one new folder implementing this interface, one registry line, migrations for any new tables, and export/test fixtures. No change to the event, registration, scoring-run or leaderboard code.

10.2 Common engine responsibilities

Responsibility	Rule
Timing	`opens`/`closes` derived from DB clock and stored `starts_at`/`ends_at`; sweeper enforces transitions but correctness does not depend on it
Eligibility	`rounds.eligibility` JSON: `all`, `qualified_from:[round_ids]`, `top_n`, `top_percent`, `min_score`, `manual_list`; evaluated server-side on every participant action (not only on page render)
Advancement	Computed at finalize from the round's scores: `top_n`, `top_percent`, `min_score`, `manual`, `all`; ties at the cutoff follow the round's `tie_policy` (`include_all`, `use_tiebreak`, `manual`); result stored in `entry_round_status`; manual override requires reason + audit
Pause/resume	Sets `paused_at`; resume inserts a `round_pauses` row and extends every in-progress attempt/challenge clock by the pause length exactly once (BR-14) inside one transaction
Reordering	Allowed freely in `draft`; after any participant activity only via a guarded flow that revalidates eligibility dependencies
Config immutability	Once a round has attempts/submissions/scores, structural config changes are blocked or require a new scoring config version and audited "rescore" (§14.7)

10.3 Judged round type

`judged` rounds have no participant upload; judges score entries live or from provided evidence (e.g., pitch, viva). Participants see instructions and status only. Scoring uses the rubric engine (§13) and produces `judge_aggregate` score records.

11. Quiz engine

11.1 Authoring and validation

- Question types: `single`, `multi`, `truefalse` (a `single` with two fixed options), `text` (optional), with optional image in stem/options (`media_assets`, mime allow-list, size limit).
- Validation on save: ⩾ 2 options for choice types; ⩾ 1 correct (exactly 1 for `single`/`truefalse`); marks > 0 and ⩽ configured maximum; negative marks ⩾ 0 and ⩽ marks; for `text`, ⩾ 1 accepted answer.
- Editing a question that any attempt has used creates a new `question_versions` row (BR-06). Attempts already started keep their `attempt_questions.question_version_id`.
- **Preview:** admin preview renders from a synthetic in-memory attempt; it writes nothing to attempt tables.

11.2 Selection and randomization

- Round quiz composition: fixed list (`round_questions.position`) and/or pools (`pool_key` + count, sampled by tag/difficulty).
- At attempt start the server generates a `seed` (stored) and produces `attempt_questions` rows (order, sampled subset, option order) using a deterministic PRNG (seeded Fisher–Yates). Order is therefore stable across refreshes and reproducible in audits.
- Correct-answer flags are **never** sent to the browser before results are released.

11.3 Attempt start (server)

Inside one transaction:

1. Authorize: the actor's participant owns the entry; entry `active`; round `open`; eligibility satisfied; event not paused/archived.
2. Serialize per entry: `pg_advisory_xact_lock(hash(round_id, entry_id, participant_id))`.
3. Enforce `max_attempts`; if an `in_progress` attempt exists, **return it** (idempotent resume) with remaining time.
4. `started_at = now()`; `deadline_at = LEAST(started_at + duration, rounds.ends_at)` (BR-04); persist.
5. Generate frozen questions; commit; respond with questions (without answers), `deadline_at`, `server_time`.

If the round window has no time left (`deadline_at <= now()`), the start is refused.

11.4 Scoring rules

All marks are stored as **scaled integers** (`marks × 10,000`) or `numeric`, never floating point `[P]`.

Type	Rule (marks `m`, negative marks `n`, default `n = m × negative_fraction`)
`single`, `truefalse`	correct → `+m`; wrong → `-n`; blank → `0`
`multi` — `all_or_nothing`	exact set match → `+m`; blank → `0`; otherwise → `-n`
`multi` — `proportional`	With `C` correct options, `sc` correct selected, `sw` wrong selected, weight `λ` (default 1): `raw = m × (sc − λ·sw) / C`, then clamp to `[-n, m]`
`text`	Normalize (NFKC, trim, collapse whitespace, case-fold; optional numeric tolerance); match against accepted list → `+m`; no match → `needs_manual_review` (excluded from auto total until reviewed; round cannot finalize with pending reviews)

Quiz total = Σ question scores; optionally floored at 0 (`floor_total_at_zero`, default false `[P]`, BR-12). Scoring is a pure function `scoreAttempt(frozenQuestions, responses, config)` used by submit, rescore and tests.

11.5 Save and submit protocol

Save answers — `PUT /api/v1/attempts/{id}/answers`

- Body: `{ answers: [{ attemptQuestionId, answer, clientSeq }], clientSentAt }` (batched, debounced ≈ 3–5 s `[T]`).
- One conditional statement: upsert only if the attempt is `in_progress` and `now() <= deadline_at + save_grace` and `clientSeq` is greater than the stored one. Rejected rows are reported individually.
- Returns `{ savedSeq, serverTime, deadlineAt, status }`. Once past the deadline: `409 attempt_closed` with the final state.

Submit — `POST /api/v1/attempts/{id}/submit` with `Idempotency-Key`

- `SELECT … FOR UPDATE` the attempt; if already `submitted`/`auto_submitted`, return the stored result (idempotent).
- Accept final answers included in the request only if before the deadline (grace `0` by default `[P]` D-P04); apply them with the same `clientSeq` rule; compute score server-side; write `quiz_attempts.score`, `score_records(source=quiz_auto)`; set `status`, `submitted_at`, `submit_source=manual`; commit; respond with confirmation (score visibility per round setting).

Auto-submit — when `deadline_at` passes with the attempt `in_progress`: finalized **lazily** by any read/write touching the attempt, and **proactively** by the sweeper (`FOR UPDATE SKIP LOCKED`, batch 200) and by "close round" (BR-24). Saved answers are scored; `submit_source=auto`. Correctness does not depend on the sweeper's schedule; only the timeliness of published scores does.

11.6 Timer behavior

The server returns `deadline_at` and `server_time` on every quiz response. The client computes an offset once (`offset = serverTime − (clientNow − rtt/2)`), refreshes it from each response, and renders `deadline_at − (Date.now() + offset)`. The countdown is purely presentational (BR-03). On reaching zero the client calls submit; whatever the server decides is final.

11.7 Refresh and network interruptions

`GET /api/v1/attempts/{id}` returns questions, saved answers, `deadline_at`, `server_time`, status — so a refresh, a new tab, or a crashed browser resumes correctly. The client keeps an ordered outbound queue (memory + IndexedDB), retries with exponential backoff and jitter, shows *Saved* / *Saving* / *Offline – retrying*, and flushes on `online`/`visibilitychange`. Duplicated deliveries are harmless (`clientSeq`).

11.8 Quiz submission workflow

```
sequenceDiagram
  autonumber
  participant B as Browser
  participant API as Next.js API
  participant RL as Upstash rate limit
  participant DB as Postgres

  B->>API: POST /rounds/{id}/attempts (Idempotency-Key)
  API->>RL: check attempt-start limit
  API->>DB: BEGIN; authorize; advisory lock (round, entry)
  DB-->>API: no in-progress attempt
  API->>DB: INSERT attempt (started_at=now, deadline_at) + frozen questions
  DB-->>API: COMMIT
  API-->>B: 201 questions (no answers), deadline_at, server_time

  loop every ~3-5 s while answering
    B->>API: PUT /attempts/{id}/answers [{q, answer, clientSeq}]
    API->>RL: check save limit
    API->>DB: conditional upsert (in_progress AND now <= deadline+grace AND seq newer)
    DB-->>API: saved rows
    API-->>B: 200 savedSeq, serverTime, deadline_at
  end

  B->>API: POST /attempts/{id}/submit (Idempotency-Key)
  API->>DB: BEGIN; SELECT attempt FOR UPDATE
  alt already submitted
    DB-->>API: stored result
  else before deadline
    API->>DB: apply final answers; score from frozen versions; INSERT score_record; UPDATE status
    DB-->>API: COMMIT
  else after deadline
    API->>DB: finalize as auto_submitted from saved answers
    DB-->>API: COMMIT
  end
  API-->>B: 200 confirmation (or 409 with final state)
```

12. Development challenge submissions

12.1 Flow

- **Fixed window** (`clock_mode=fixed_window`): all entries share `rounds.starts_at`/`ends_at`.
- **Per-entry clock** (`per_entry_clock`): entry presses **Start challenge**; `challenge_starts.deadline_at = LEAST(now() + time_limit, rounds.ends_at)` stored once (idempotent).
- **Draft** (`PUT …/draft`): stores `submission_revisions` with `is_draft=true`; never counted, never scored, overwritten in place.
- **Submit** (`POST …/submit`, `Idempotency-Key`): validate → check deadline via DB clock → insert `submission_revisions(revision_no = max+1)` in a transaction that locks the `submissions` row; update `counted_revision_id` according to the rule (D-P07: latest on-time). Response includes `revision_no`, `submitted_at` (server), and status.

```
sequenceDiagram
  autonumber
  participant B as Browser
  participant API as Next.js API
  participant DB as Postgres
  B->>API: PUT /rounds/{id}/submission/draft (optional, autosave)
  API->>DB: upsert draft revision (not counted)
  B->>API: POST /rounds/{id}/submission (Idempotency-Key, URLs, description)
  API->>API: Zod + URL validation (no network calls)
  alt invalid
    API-->>B: 422 field errors
  else valid
    API->>DB: BEGIN; lock submissions row; check deadline vs now()
    alt before deadline
      API->>DB: INSERT revision n+1; set counted_revision; COMMIT
      API-->>B: 201 revision_no, submitted_at, status=submitted
    else after deadline
      Note over API,DB: per late policy: reject, accept with penalty flag, or accept flagged
      API-->>B: 409 late or 201 with is_late=true
    end
  end
  Note over B: retry with same key returns the same revision
```

12.2 URL validation (no server-side fetching in the MVP)

Check	Rule
Parse	WHATWG `URL`; normalize host to lower case; drop fragment
Scheme	`https` (and `http` only if the event allows) — never `javascript:`, `data:`, `file:`
Length	≤ 2,048 characters
Userinfo	Reject `user:pass@`
Host	Must be a registrable domain; reject IP literals, `localhost`, `*.local`, `*.internal`, and private/link-local ranges
GitHub field	If `repo_url` must be GitHub: `https://github.com/<owner>/<repo>` with valid owner/repo characters
Video field	Optional host allow-list (e.g., YouTube, Loom, Google Drive) per event
Duplicates	Same normalized `repo_url` across entries in the event → **flag** for organizer review (collusion signal), not a hard error
Display	Rendered with `rel="noopener noreferrer nofollow ugc"` and `target="_blank"`; shown as text with hostname emphasized

Ownership (BR-18): the UI states that links are participant claims. Verified ownership would need an OAuth/GitHub App integration (post-MVP `[P]`); until then organizers use the manual verification checklist in the judging UI. A later optional link-check job must use SSRF protections (public-IP-only DNS resolution, redirect limit, short timeouts, no credentials, response-size cap).

12.3 Status derivation

`not_started` → `draft` → `submitted` (on time) / `submitted_late` → `under_review` (assigned) → `evaluated` (final scores locked).
Withdrawn/disqualified entries show their entry status instead.

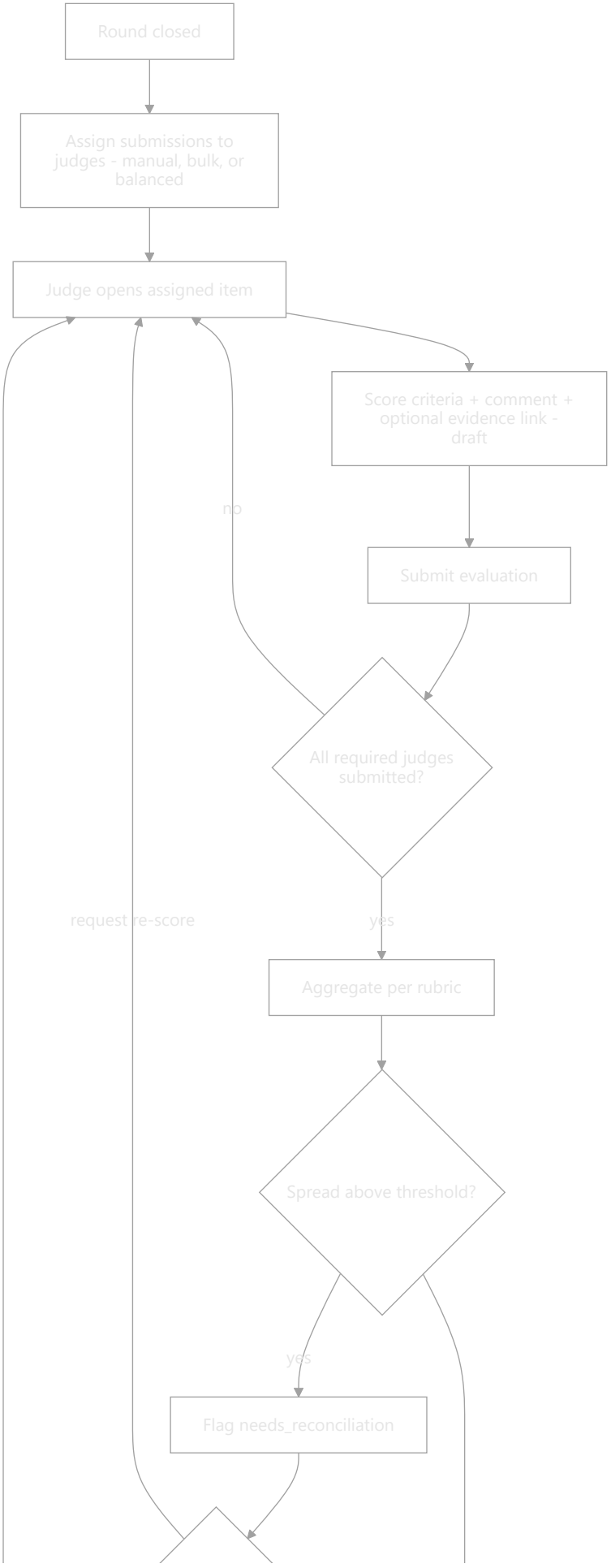
13. Manual judging

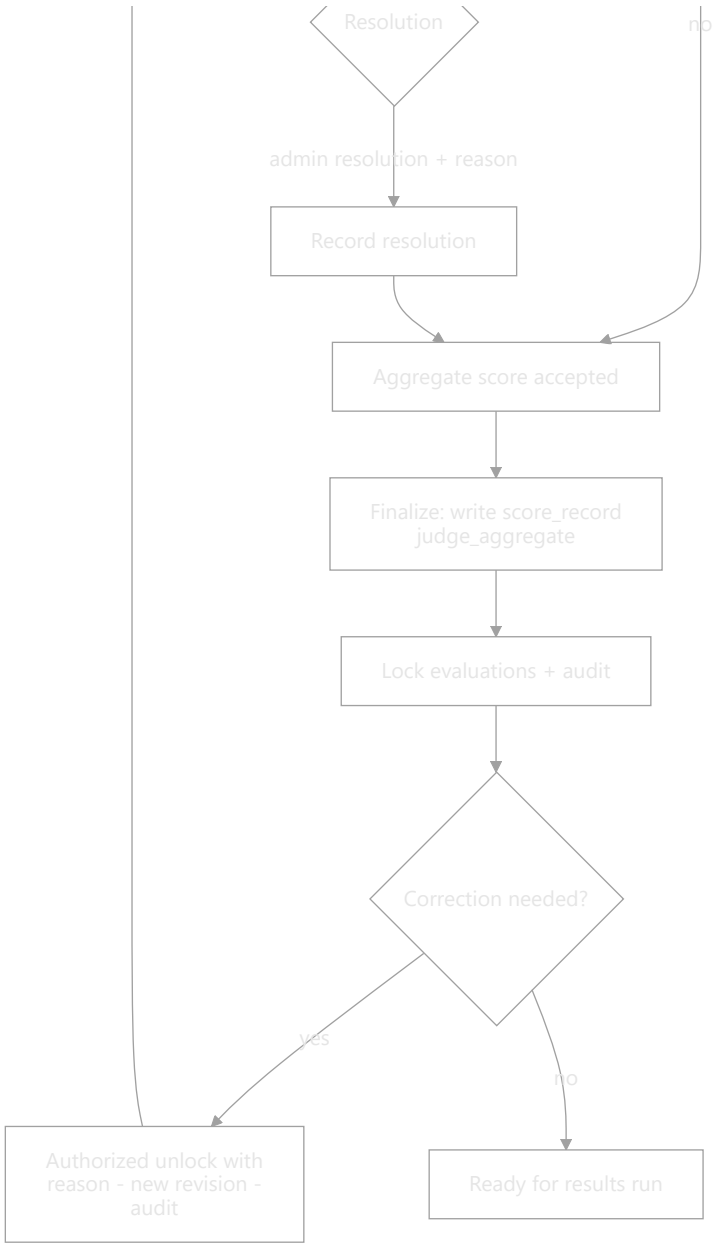
13.1 Rubric and evaluation model

- **Rubric** per round: criteria with `max_marks`, optional `weight`, description; aggregation method; variance threshold; `min_judges`; `blind`.
- **Judge total** for one evaluation: without weights `total = (Σ marks / Σ max) × round_max`; with weights `total = (Σ (marks_c / max_c) × w_c / Σ w_c) × round_max`.
- **Aggregate across judges:** `mean`, `median`, `trimmed_mean` (drop `trim_pct` from each end), `max`, `min`, or `weighted_judges` (`judging_assignments.judge_weight`).
- **Disagreement:** `spread = (max_total − min_total) / round_max × 100`. If `spread > variance_threshold_pct` the item becomes `needs_reconciliation`.
- **Resolution (authorized roles):** (a) request re-score (judge submits a new revision), or (b) record a resolution with method (`adopt_mean`, `adopt_median`, `manual_value`), value and mandatory reason in `evaluation_resolutions`.

- **Conflicts of interest:** a judge who declares a conflict is excluded by the assignment validator; the assignment optimizer enforces `min_judges` and load balance.
- **Blind mode:** judges see a pseudonymous item code and no team/participant names.

13.2 Workflow


Round closed
Assign submissions to judges - manual, bulk, or balanced
Judge opens assigned item
Score criteria + comment + optional evidence link - draft
Submit evaluation
All required judges submitted?
no
request re-score
yes
Aggregate per rubric
Spread above threshold?
yes
Flag needs_reconciliation



13.3 Audit and locking

Every evaluation save creates a revision row; every state change (submit, resolve, finalize, unlock) writes an `audit_logs` entry with actor, timestamp, before/after, and reason (FR-JDG-07). Locked evaluations are immutable (BR-08); unlocking creates a new revision and a **superseding** score record — history is never rewritten.

14. Scoring and results

14.1 Principles

1. **Raw scores** (`score_records`) are append-only facts with their scoring-config version.
2. **Results** (`result_runs` + `calculated_results`) are derived, versioned and reproducible.
3. No update ever silently changes a published result: corrections create new score records and a new run; the old run remains addressable (BR-09, BR-11).

14.2 Configuration

- **Round scoring config** (`round_scoring_configs`, immutable versions): quiz options (marks scale, negative marking, partial-credit mode, floor), late penalties, rubric reference, normalization.
- **Event result config** (`events.result_config`, versioned by `result_config_version`, always copied into `result_runs.config_snapshot`):

```
json
```

```
{
  "mode": "weighted",
  "scale": 100,
  "rounds": [
    { "round_id": "…", "weight": 0.3, "include": true },
    { "round_id": "…", "weight": 0.7, "include": true }
  ],
  "missing_round_policy": "zero",
  "penalties": { "cap": 20 },
  "tie_break": ["round_score:<round_id>", "total_time_asc", "last_submit_asc"],
  "rank_scope": "active_only"
}
```

14.3 Calculation (pure function)

`computeResults(inputs, config) → rows` is a deterministic function (no clock, no randomness, no I/O). For each entry and included round:

```
base        = the single non-superseded primary record (quiz_auto | judge_aggregate | quiz_manual)
adjustments = Σ admin_adjust (non-superseded)
penalties   = Σ penalty (non-superseded, positive numbers)
round_final = base + adjustments − penalties                (optional floor at 0)
contribution (mode=weighted) = weight × (round_final / round_max) × scale
contribution (mode=sum)      = round_final
final_score = Σ contributions − event_level_penalties
```

A partial unique index guarantees at most one non-superseded primary record per (round, entry). Fixed-point arithmetic is used; results are rounded once at the end with a documented rule (round half up to 4 decimals `[P]`).

14.4 Special statuses

Situation	Storage	Ranking	Display / export
Incomplete (attempt started, not submitted)	Auto-submitted from saved answers, `submit_source=auto` (BR-24)	Ranked normally	Flag `auto_submitted`
Unscored round (no attempt/submission/evaluation)	No primary record	Per `missing_round_policy`: `zero` (score 0, `result_status=incomplete`), `exclude` (unranked), `rank_last`	`unscored` distinct from `0`
Withdrawn (BR-20)	Records retained	`rank = null`	`withdrawn`
Disqualified (BR-21)	Records retained, reason stored	`rank = null`	`disqualified` + reason
Team member change (BR-22)	Team scores unchanged unless audited correction	—	—
Provisional run	`status=provisional`	Ranked	Labelled *Provisional*
Final run	`status=final`, published	Ranked	Labelled *Final*; run number shown

14.5 Ranking and ties

Sort by `final_score` descending, then each element of the `tie_break` chain in order. Entries with identical complete keys share a rank; the next rank skips (1, 2, 2, 4 — BR-25). `tie_status`: `none`; `tied_resolved` (a tie-break separated equal totals); `tied_unresolved`. Unresolved ties at an advancement cutoff follow the round's `tie_policy`.

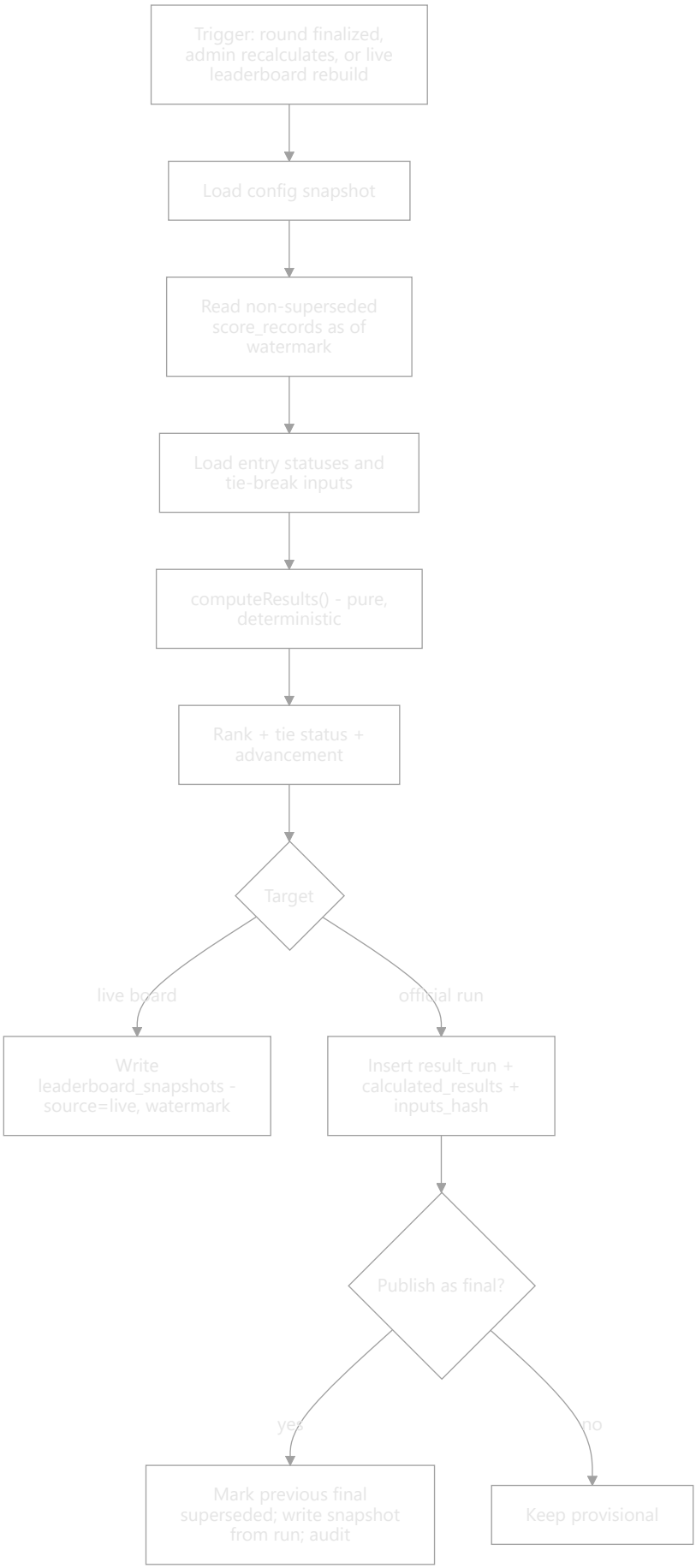
14.6 Reproducibility

`result_runs` stores `config_snapshot` (event result config + every round scoring config version used) and `inputs_hash = SHA-256(canonical JSON of config + score-record ids/values valid as of the run + entry statuses + tie-break inputs)`. "Valid as of" uses `created_at <= run.created_at AND (superseded_at IS NULL OR superseded_at > run.created_at)`. Recomputing a past run from stored data must reproduce byte-identical `calculated_results` (tested with golden fixtures, T-9.6).

14.7 Corrections and rule changes

Change	Handling
Correct a judge evaluation	Unlock (authorized, reason) → new evaluation revision → new `judge_aggregate` record **supersedes** the old one → new provisional run
Manual adjustment/penalty	New `admin_adjust`/`penalty` record with reason and audit
Question found wrong after the quiz	Admin creates a corrected question version + **rescore action**: dry-run diff first; on confirm, new `quiz_auto` records supersede old ones (config version noted); never edits responses
Scoring rule change	New immutable config version; existing records keep their version; an explicit rescore is required to apply it; UI marks the current final run **stale** until re-run
Publishing a correction	New final run supersedes the earlier final run; participants see "corrected on …"; exports include `result_run_no` and `result_status`

14.8 Calculation workflow


Trigger: round finalized, admin recalculates, or live leaderboard rebuild
Load config snapshot
Read non-superseded score_records as of watermark
Load entry statuses and tie-break inputs
computeResults() - pure, deterministic
Rank + tie status + advancement
Target
live board
official run
Write leaderboard_snapshots - source=live, watermark
Insert result_run + calculated_results + inputs_hash
Publish as final?
yes
no
Mark previous final superseded; write snapshot from run; audit
Keep provisional

15. Leaderboard

15.1 Visibility policy

`events.leaderboard_policy` (and optional per-round overrides) — evaluated on the server on every read:

Mode	Who can read
`hidden_until_end`	Staff only until event status `completed`, then per `post_event_audience`
`during_rounds`	Audience allowed only while rounds listed in `visible_round_ids` are `open`/`closed`/`published`
`staff_only`	Super admin, event admin, round manager, judges (if allowed), reviewers
`participants_only`	Authenticated participants of the event + staff
`public`	Anyone

Optional flags: `show_own_rank_when_hidden`, `freeze_from` (timestamp after which non-staff see the last pre-freeze snapshot), `display_fields` (name, team, public code — never email/phone), `top_n`.

15.2 Update strategy

- **Live board = derived snapshot, not per-submission broadcast.** Score commits do **not** write a shared hot row. A snapshot is stale when `now() − generated_at ≥ min_rebuild_interval` (default 5 s `[T]`) **and** `max(score_records.id)` for the event exceeds the snapshot's `watermark`.
- **Single-flight rebuild:** the first request/tick that finds the snapshot stale acquires `pg_try_advisory_xact_lock(hash('lb', event_id, scope))`, recomputes with `computeResults` over ≈ 500 entries, inserts `version = prev + 1`, commits. Everyone else keeps serving the previous snapshot (stale-while-revalidate).
- **Triggers:** stale-check on leaderboard reads, the tick job during live rounds, and admin publish/finalize actions.
- **Official vs live:** while an event runs, the board is labelled *Provisional* (`source=live`). When a round or the event result is published, snapshots come from the **result run** (`source=run`) and match exports exactly.
- **Payload:** top-N (default 100) rows ≈ 8 KB raw / ≈ 3 KB gzip `[E]`, plus `version`, `generated_at`, `provisional`, `etag`. Own rank is a separate small authenticated call (`/leaderboard/me`, single indexed lookup).

15.3 Delivery

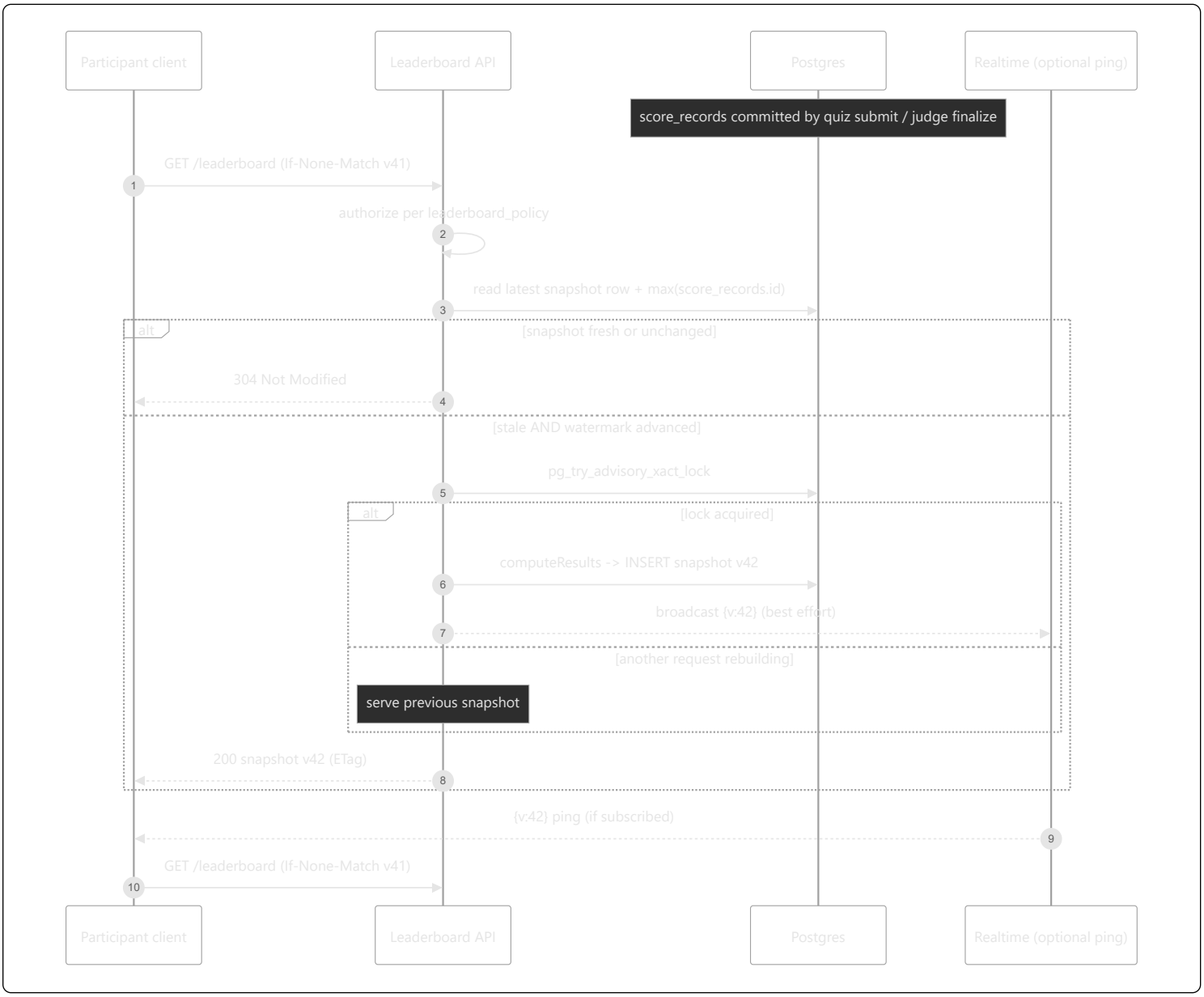
Path	Details
Baseline (MVP): polling	`GET /api/v1/events/{id}/leaderboard?scope=…` with `If-None-Match: "<scope>-<version>"` → `304` when unchanged. Client interval 15 s default (min 5 s) with ±20% jitter, paused when the tab is hidden, backs off to 30 s after repeated 304s `[T]`
Caching	Public boards: `Cache-Control: public, s-maxage=5, stale-while-revalidate=10` `[T]`. Restricted boards: `private, no-cache` (authorization must run every request)
Optional: realtime ping	Supabase Realtime Broadcast on a **private** channel `event:{id}:leaderboard` carrying only `{ v: <version> }`; clients then fetch the snapshot. Subscription authorization via Realtime Authorization policies `[U – verify]`. Ping rate ⩽ 1 per 10 s per event keeps fan-out at ≈ 50 msg/s for 500 subscribers, inside the Free 100 msg/s cap but consuming connections `[V]`. MVP uses it for staff dashboards; participant realtime is a later toggle
Failure fallback	On `too_many_connections`, errors or disconnect, the client silently reverts to polling

15.4 Reconnection and consistency

State is **version-based, not event-based**: after any interruption (offline, sleep, tab restore) the client refetches the latest authorized snapshot; there is nothing to replay. The UI shows *last updated* and a stale banner if data is older than 2× the poll interval.

15.5 Protection

Authorization precedes data access on every read (`403` reveals nothing beyond "not available"); snapshot rows never contain PII columns; private realtime channels are authorized by the same policy function; frozen/hidden states are enforced server-side, so client tampering cannot reveal scores.



16. CSV export

16.1 Export types

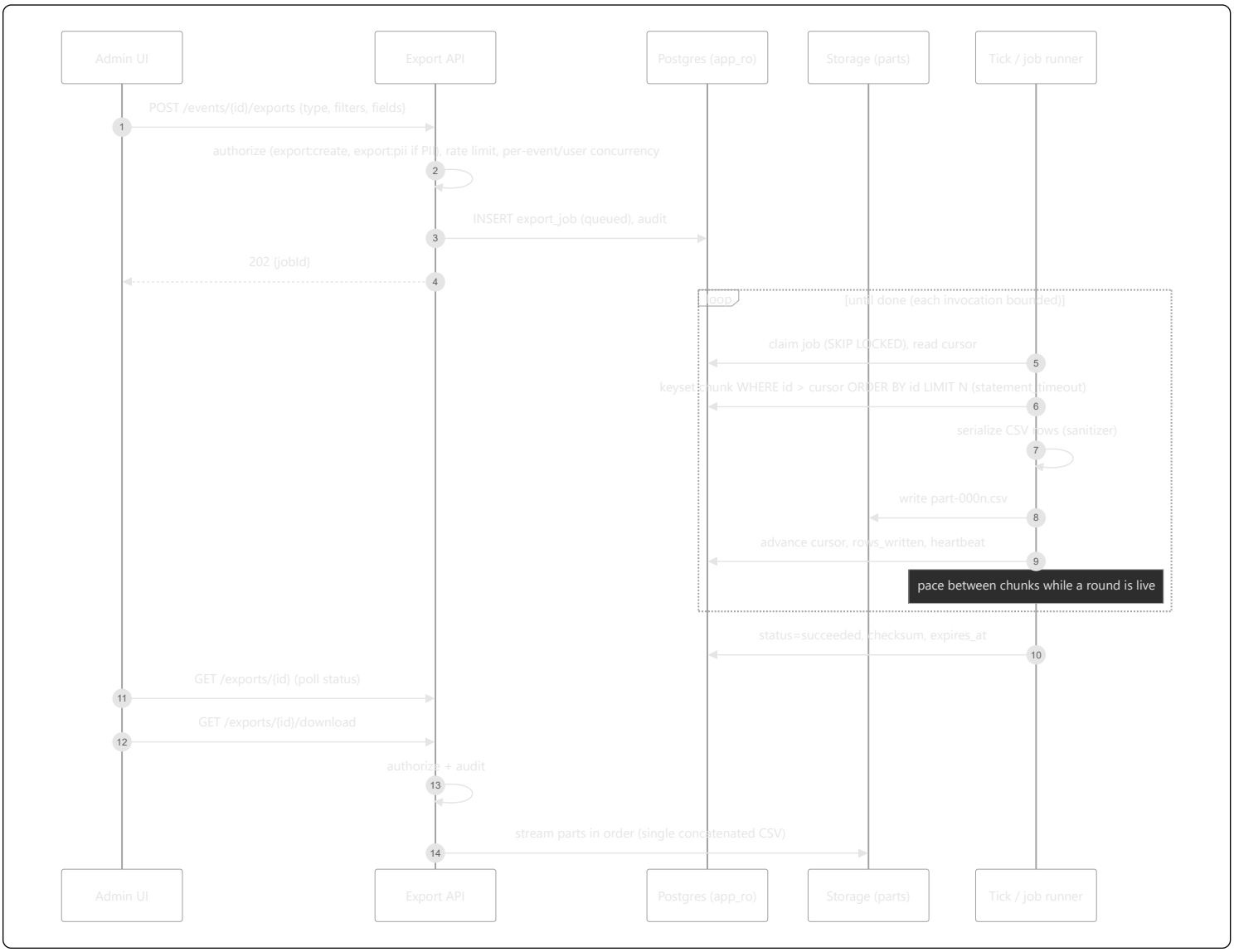
Type	Source	Notes
`registrations`	`participants`, form field defs, teams	PII fields require `export:pii` (BR-27); configurable fields/filters (status, team, date)
`quiz_responses`	`quiz_responses` ⋈ `attempt_questions` ⋈ `question_versions`	One row per answered question (long) with `attempt_id`, entry/participant IDs, question version, given answer, correct flag, awarded marks; optional attempt-summary variant
`challenge_submissions`	`submission_revisions`	All revisions with `counted` flag, URLs, timestamps, late flag
`evaluations`	`judge_evaluations` + scores	One row per judge × criterion; comments; evidence link; aggregate & resolution columns
`round_results`	`score_records`/`calculated_results` for one round	Round score, max, penalties, status, advancement
`event_results`	`calculated_results` of a **specific run**	Wide (default) or long format

Final results columns (FR-EXP-02): `event_id, event_slug, event_title, result_run_no, result_status, entry_code, entry_kind, participant_codes, team_name, display_name`; per round in sequence (columns generated from rounds — BR-30): `round_<n>_title, round_<n>_score, round_<n>_max, round_<n>_status`; then `raw_total, weighted_total, penalties, final_score, final_rank, tie_status, advancement_status, submission_links, entry_status, status_reason`.

16.2 CSV format rules

Aspect	Rule
Standard	RFC 4180: comma separator, CRLF line endings, fields with comma/quote/CR/LF quoted, quotes doubled
Encoding	UTF-8; optional BOM (default on, for spreadsheet compatibility `[P]`)
Types	Numbers typed as numbers (`.` decimal, no thousands separators); booleans `true`/`false`; timestamps ISO-8601 UTC (`2026-09-25T10:15:30Z`) plus optional event-local column
Identifiers	Same `entry_code`, `participant_code`, `team` IDs in every export
Formula-injection protection (BR-28)	For every **string-typed** cell whose first character is `=`, `+`, `-`, `@`, TAB (0x09) or CR (0x0D), prepend a single quote `'` before quoting. Numeric columns come from numeric types and are never string-sanitized, so legitimate negatives (`-2.5`) are unaffected. The sanitizer is one shared function with a fixed payload corpus test
Schema stability	`schema_version` in the manifest; header order fixed per type/version
Manifest	Sidecar JSON: filters, fields, requester, `exported_at`, row count, SHA-256, `result_run_no`

16.3 Execution model (does not block ordinary traffic)



Controls: **fast path** — exports under a small row threshold (≈ 5,000 `[T]`) stream directly in one request through the same serializer; **concurrency** — ⩽ 2 running per event, 1 per user, global cap 3 `[T]`; **chunking** — 1,000 rows/chunk (250 for wide formats); **pacing** — configurable delay between chunks while any round in the event is `open` (default 150 ms `[T]`); **isolation** — read-only DB role with `statement_timeout` (5 s) and `lock_timeout` (1 s); **resumability** — cursor and part list persist, so a restart or failed invocation continues from the last completed chunk; stale jobs (no heartbeat) are re-queued up to 3 times; **retention** — parts and files expire (default 24 h `[P]`) and are cleaned by a job; **consistency** — results exports read one immutable result run; registration/response exports are live data stamped with `exported_at`.

17. API design

17.1 Conventions

- REST-style JSON under `/api/v1`; all inputs validated with Zod; unknown fields rejected.
- **Errors:** `{ "error": { "code": "attempt_closed", "message": "…", "details": {…}, "requestId": "…" } }` with correct HTTP status (400/401/403/404/409/422/429/5xx). No stack traces or SQL in responses.
- **Idempotency:** header `Idempotency-Key` (client UUID) required on start/submit/registration/export-create; retried keys return the original response.
- **Pagination:** cursor (`?cursor=…&limit=…`, max 100); stable sort keys.
- **Time:** every timing-sensitive response includes `serverTime`.
- **Rate limits (Upstash), all `[T]`, tuned by LT-*:**

Class	Limit
auth sign-in	10/min/IP and 5/min/account
registration submit	5/min/user, 30/min/IP
attempt start	5/min/user
answer save	burst 10, sustained 30/min/attempt
quiz/challenge submit	10/min/entry
leaderboard read	ETag + short cache; abuse guard at the edge `[U Vercel Firewall]` — **not** through Redis (see §4.3)
export create / download	3/hour/user and 10/hour/user
admin mutations	60/min/user

17.2 Endpoint catalogue (proposed)

Auth/session

Method & path	Auth	Notes
`POST /auth/sign-up`, `/auth/sign-in`, `/auth/sign-out`, `/auth/reset`	public / session	Supabase Auth adapter
`GET /me`	user	profile + role assignments

Events and public

Method & path	Permission	Notes
`GET /events`, `GET /events/{slug}`	public	published only
`POST /events`	`event:create`	
`PATCH /events/{id}`	`event:update`	
`POST /events/{id}/publish` `/pause` `/resume` `/complete` `/archive`	`event:lifecycle`	audited transitions
`PUT /events/{id}/registration-form`	`event:update`	new form version
`PUT /events/{id}/roles`	`event:assign_roles`	

Registration and teams

Method & path	Permission	Notes
`POST /events/{id}/registrations`	participant	idempotent; capacity-safe
`GET/PATCH /events/{id}/registrations/me`	owner	until lock
`GET /events/{id}/registrations`	`registration:read`	search/filter/paginate
`POST /events/{id}/registrations/{pid}/review`	`registration:review`	approve/reject/waitlist
`POST /events/{id}/teams`, `POST /teams/{id}/join`, `POST /teams/{id}/leave`, `PATCH /teams/{id}`	participant/captain	invite-code join

Rounds

Method & path	Permission	Notes
`POST/PATCH/DELETE /events/{id}/rounds…`, `POST /events/{id}/rounds/reorder`	`round:manage`	
`POST /rounds/{id}/open` `/close` `/pause` `/resume` `/extend`	`round:control`	audited; extend needs reason
`PUT /rounds/{id}/scoring-config`	`round:manage`	creates new immutable version
`GET /events/{id}/me/rounds`	participant	eligibility-aware list

Quiz

Method & path	Permission	Notes
`POST/PATCH /events/{id}/question-banks…`, `…/questions…`	`quiz:author`	versioning
`POST /rounds/{id}/quiz/preview`	`quiz:preview`	no persistence
`POST /rounds/{id}/attempts`	participant	start/resume; idempotent
`GET /attempts/{id}`	owner	resume state
`PUT /attempts/{id}/answers`	owner	batched save
`POST /attempts/{id}/submit`	owner	idempotent
`POST /attempts/{id}/invalidate`	`round:control`	admin

Challenge

Method & path	Permission	Notes
`PUT /rounds/{id}/challenge`	`challenge:author`	task definition
`POST /rounds/{id}/challenge/start`	participant	per-entry clock
`PUT /rounds/{id}/submission/draft`	owner	not counted
`POST /rounds/{id}/submission`	owner	idempotent revision
`GET /rounds/{id}/submission`	owner	status + revisions
`GET /rounds/{id}/submissions`	`submission:read`	staff list

Judging

Method & path	Permission	Notes
`PUT /rounds/{id}/rubric`	`rubric:manage`	
`POST /rounds/{id}/assignments` (auto/bulk/manual)	`judging:assign`	COI checks
`GET /judge/assignments`	`judging:score`	own only
`PUT /assignments/{id}/evaluation`	`judging:score`	draft/submit
`POST /rounds/{id}/reconcile`	`judging:reconcile`	reason required
`POST /rounds/{id}/finalize`, `POST /rounds/{id}/unlock`	`scores:finalize`, `scores:unlock`	audited

Results and leaderboard

Method & path	Permission	Notes
`POST /events/{id}/results/runs`	`results:publish`	dry-run or provisional
`POST /events/{id}/results/runs/{n}/publish`	`results:publish`	final
`GET /events/{id}/results`	per policy	latest published
`GET /events/{id}/leaderboard`, `GET /events/{id}/leaderboard/me`	per policy	ETag
`PUT /events/{id}/leaderboard-policy`	`leaderboard:configure`	

Exports, admin, audit

Method & path	Permission	Notes
`POST /events/{id}/exports`, `GET /exports/{id}`, `GET /exports/{id}/download`	`export:create`	audited
`GET /events/{id}/dashboard`	staff	counts, active rounds, pending evaluations
`GET /events/{id}/audit`	`audit:read`	filter/search
`GET /rounds/{id}/monitor`	`round:control`	live attempt counts

Internal (not user-callable)

Method & path	Auth	Notes
`POST /internal/jobs/tick`	`CRON_SECRET` bearer + allow-list	sweeper, email queue, export chunks, cleanup

---

18. Scalability analysis

Everything in this section is an **estimate** `[E]` **or target** `[T]`. No load test has been run. The purpose is to identify where the system is most likely to break and what to measure first.

18.1 Workload model (assumptions to be confirmed by the owner)

Parameter	Assumption
Active participants `N`	500 individuals (headroom test to 750–1,000); teams of ⩽ 4 reduce challenge submissions to ≈ 125–250
Quiz	30 questions, 30-minute attempt window, ≈ 90% of participants start within 60 s of opening
Answer behavior	≈ 1 answer/minute; debounced autosave every ≈ 3–5 s while changing answers ⇒ ≈ 35 saves per participant per quiz
Deadline behavior	Up to 500 submits within 3 s if the client confirms exactly at the deadline (worst case)
Leaderboard	All 500 viewing for 3 hours, polling every 15 s ⇒ 4 requests/min/viewer
Challenge	500 entries (or teams) submit within the last 60 s of a shared deadline; ≈ 10% retry
Admin	10 staff; monitor pages polling every 5–10 s

Derived volumes for one event day `[E]`: quiz ≈ 500 × 40 ≈ **20,000 API calls**; challenge ≈ 5,000; leaderboard ≈ 500 × 4 × 180 ≈ **360,000**; pages/assets/admin ≈ 25,000 ⇒ **≈ 410,000 function invocations** and ≈ **15,000–45,000 answer row-writes** (final rows vs. worst-case autosave upserts).

18.2 Peak scenarios

#	Scenario	Rate	Where it hurts	Design response
S1	**500 simultaneous active users** (mixed)	≈ 30–60 rps steady	Auth/session verification cost; function cold starts [U]	Local JWT verification [U]; pre-warm before rounds (synthetic requests); thin route handlers
S2	**500 quiz starts in 30 s**	≈ 17 rps avg, ≈ 100 rps burst	Per-entry advisory locks, question generation, payload size (≈ 60 KB raw per attempt)	Short transaction; gzip; images from CDN; pre-generated question payload cache per round [P]
S3	**500 submits in 3 s**	≈ 167 rps peak	DB CPU on shared compute; pooler client cap; single-row `FOR UPDATE` per attempt (no cross-attempt contention)	Autosave makes submit a small delta; scoring is O(30 rows); **deadline-aligned surge damping**: saved answers are auto-submitted server-side, so the client's confirm call at T=0 carries **random 0–3 s jitter** and is idempotent
S4	**500 leaderboard viewers**	≈ 33 rps × 2 tiny queries	Auth per request; snapshot read; egress ≈ 100 KB/s ≈ 1.1 GB per 3 h	ETag/304; snapshot row; public boards CDN-cached; polling rather than 500 WebSockets [V limits]
S5	**Challenge deadline surge** (500 in 60 s)	≈ 8 rps avg, ≈ 50 rps burst	Duplicate/retry storms; validation CPU	Idempotency keys; no outbound fetches; lightweight validation; per-entry row lock only
S6	**Registration opening** (500 in 60 s, capacity 400)	≈ 8 rps	Serialization on the event row for capacity/counter	Lock held for ≈ 10 ms; measured throughput ≈ 100/s theoretical [E]
S7	**Export during a live round**	1–2 background streams	Read load and lock contention	Read-only role, keyset chunks, pacing, concurrency caps

18.3 Bottleneck analysis by layer

Layer	Likely bottleneck	Evidence/limit	Mitigation	Measure with
Frontend	Large JS bundle on mobile; image weight; timer re-renders	—	Route-level splitting; budget ⩽ 170 KB gz JS on quiz route [T]; system fonts; compress/limit images (⩽ 100 KB)	Lighthouse CI, WebPageTest
Backend (functions)	Burst concurrency 1,000/10 s/region [V]; cold starts [U]; function-invocation quotas [S]	500 concurrent < 1,000 burst	Keep handlers small; avoid heavy imports; pre-warm; paid plan for events	k6 LT-2/LT-4
Database CPU	Shared vCPU compute on Micro/Small [V]	≈ 167 rps × ≈ 7 ms CPU ≈ 1.2 cores [E]	Small/Medium compute for events; efficient SQL; batch inserts	DB CPU % in Supabase reports [V]
DB connections	Pooler clients: Micro 200, Small 400, Medium 600 [V]; concurrency ≈ rps × latency (Little's law)	167 rps × 0.3 s ≈ 50; 167 rps × 1 s ≈ 167	Small/Medium; `DB_POOL_MAX=1–2` per instance; short txns; no long idle-in-transaction	Pooler/connection charts
Locks	Registration (event row); attempt start advisory lock; leaderboard rebuild advisory lock	Per-entry, not global	Keys are per entry/event/scope; no hot shared row on submit	`pg_stat_activity`, lock wait metrics
Network	Mobile-network jitter and retransmissions	—	Small payloads; retry with jitter; idempotency	Client telemetry
Auth	Supabase Auth rate limits; email caps	100–300 emails/day [V]; auth limits [S]	Optional/staged verification; custom SMTP; pre-authenticated sessions before round start; no per-request Auth-server calls	LT-1, registration test
Real-time	Free: 200 conns, 100 msg/s [V]; Pro: 500 conns	500 viewers > 200	Polling baseline; ping optional; staff-only realtime in MVP	Realtime reports
Storage/egress	Image traffic	Supabase Free cached egress 5 GB [V]	Optimize images; long cache TTL	Egress dashboards

18.4 Connection math (example, [E])

`concurrent requests ≈ arrival rate × mean latency`. At the S3 peak of 167 rps with 0.3 s latency ≈ 50 concurrent requests → ≈ 50–100 pooler client connections (pool max 1–2 per instance). Micro (200 pooler clients [V]) is adequate on paper only if latency stays low; if latency degrades to 1 s, concurrency reaches ≈ 167 and Micro is at 83% of its cap. **Recommendation:** run events on **Small (400)** at minimum and test Medium (600); the concurrent *database* connections needed (≈ 5–10 active transactions) are far below the direct-connection caps (60/90) [E].

18.5 Measurable performance targets [T]

Metric	Target	Scenario
Public page load (LCP, p75, mid-range mobile, 4G profile)	⩽ 2.5 s	Lighthouse/WebPageTest
Participant dashboard first meaningful content	⩽ 2.0 s p75	LT-1
Answer-save acknowledgement	p95 ⩽ 500 ms; p99 ⩽ 1.5 s	LT-3 (500 VUs)
Attempt start	p95 ⩽ 1.2 s	LT-2
Quiz submit acknowledgement	p95 ⩽ 1 s; p99 ⩽ 2.5 s	LT-4
Challenge submit acknowledgement	p95 ⩽ 1 s; p99 ⩽ 2.5 s	LT-6
Leaderboard read	p95 ⩽ 300 ms (304 or 200)	LT-5
Leaderboard freshness after score commit	⩽ 15 s p95	LT-5 + probe
Admin dashboard/list pages	p95 ⩽ 1.5 s	LT-7
CSV export (10,000 rows)	complete ⩽ 60 s under LT-7 traffic	LT-7
5xx rate (excl. intentional 429)	< 0.5%	all
Lost acknowledged writes	0	reconciliation after each test
DB CPU p95 during peak	< 80%	all
Pooler clients at peak	< 70% of plan cap	all

18.6 Can the design carry 500? Honest status

- **On paper**, the write path (small transactions, single-row locks, snapshot reads) is well within what a small Postgres instance handles; the largest open risks are shared-CPU headroom, pooler client caps under latency spikes, and serverless cold starts. **This is unproven** until LT-2…LT-8 pass on the exact event-tier configuration.
- **Free-tier-only:** not verifiable as adequate (§4.4).
- Headroom: tests at 750 and 1,000 concurrent identify the "knee"; the platform is not declared ready for 500 unless it passes 500 with ⩾ 50% headroom or the owner accepts a lower verified ceiling.

19. Security

19.1 Threat model (summary)

Threat	Example	Primary controls
Unauthorized access	Participant reads another event's data; judge sees unassigned items	Server-side authz on every route; scoped roles; composite FKs; RLS; isolation tests
Timer/score tampering	Editing client timer or posting a score	BR-02/03/05; server computes; client values ignored
Answer replay/duplication	Double submit, retry storms	Idempotency keys, unique constraints, `clientSeq`
Injection	SQL, XSS via descriptions/comments, CSV formula injection	Parameterized queries; React escaping + HTML sanitization; CSV sanitizer (BR-28)
SSRF / malicious links	Submitting internal URLs; phishing links	No server-side fetch in MVP; URL validation; safe link rendering
Abuse/DoS	Registration spam, answer flooding, export spam	Rate limits, CAPTCHA option, concurrency caps, cost caps (spend cap, Upstash budget)
Credential attacks	Stuffing, brute force	Rate limits, lockout, MFA for admins, leaked-password checks where available `[V: Pro-only on Supabase]`
Data leakage	PII in exports/logs/leaderboards	Field-level PII flags, `export:pii`, log redaction, snapshot column allow-list
Insider misuse	Admin edits scores silently	Append-only records, audit log, reasons, two-step unlock
Supply chain	Malicious dependency	Lockfile, dependency review, minimal deps, CI audit

19.2 Controls

Area	Requirement
Authentication	Supabase Auth; HTTP-only Secure SameSite=Lax cookies; MFA for admin roles `[P]`; password policy aligned with NIST SP 800-63B principles (length over composition rules; block common passwords) `[P]`; reset links single-use and short-lived
Authorization	Central `authorize()`; route registry test enforces deny-by-default; resource checks (owner/assignment/scope/status/time) after permission check
Input validation	Zod at every boundary; size limits; reject unknown fields; server-side even when client validates
Output encoding	React default escaping; user Markdown rendered with an allow-list sanitizer; no `dangerouslySetInnerHTML` without sanitized input; CSV via sanitizer
Rate limiting & abuse	Classes in §17.1; fail-open for participant hot paths with an in-memory fallback limiter and **fail-closed** for auth and export; optional Turnstile CAPTCHA on registration `[U/P]`
Sessions	Rotation on privilege change; revoke on password change/role removal; idle and absolute timeouts configured `[U — Supabase session settings availability by plan; some session controls are Pro-only per pricing table]`
Secrets	Only in Vercel/GitHub/Supabase secret stores; never `NEXT_PUBLIC_*` for secrets; service-role key server-only and used sparingly; `.env.example` has names only; secret scanning in CI; rotation runbook
CORS	No cross-origin API access: same-origin only; never `Access-Control-Allow-Origin: *` on credentialed routes
CSRF	SameSite=Lax cookies + verify `Origin`/`Sec-Fetch-Site` on state-changing requests + JSON content-type requirement; no state change on `GET`
Headers	CSP with nonces (`default-src 'self'`; `script-src 'self' 'nonce-…'`; `connect-src` limited to app and Supabase hosts; `frame-ancestors 'none'`; `base-uri 'self'`; `form-action 'self'`), HSTS, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, minimal `Permissions-Policy`
External links	`rel="noopener noreferrer nofollow ugc"`; only http(s); show hostname; never auto-fetch
Uploads	Images only; mime sniffing + extension allow-list; size cap; re-encode where feasible; separate bucket; no execution
Audit logging	Append-only, same-transaction writes for sensitive actions, includes actor/role/target/before-after/reason/IP hash/request ID; readable only via `audit:read`
Privacy	Collect only needed fields; PII flags on fields; PII excluded from leaderboards/logs; retention/deletion policy `[P]` D-P09; legal review of applicable privacy law recommended (not legal advice)
Logging hygiene	No passwords, tokens, or full emails in logs; hashed user IDs; request IDs
Dependency hygiene	Lockfile committed; scheduled dependency audit; pinned major versions; review of new dependencies

20. Reliability and failure handling

No availability figure is claimed. Behavior below is **design intent** to be proven by the recovery tests (REC-*) in §24.

Failure	Detection	System behavior	Participant experience	Recovery / rule	Test
Temporary DB outage	Query errors/timeouts; health probe	Writes return `503 Retry-After`; nothing is acknowledged unless committed (NFR-REL-01)	Answers queued locally, banner "Reconnecting — your answers are kept on this device"	Client retries with backoff. If the outage overlaps a deadline, admin applies an audited extension (BR-15) for affected entries	REC-01
Participant network interruption	Failed fetch / `offline` event	Server unaffected	Save-state indicator; queue in memory + IndexedDB	Flush in order on reconnect; `clientSeq` prevents regression	REC-02
Duplicate/repeated requests	Same `Idempotency-Key` or unique violation	Return original result; no double scoring	Same confirmation	Natural keys + idempotency table	REC-03
Browser refresh / new device	—	`GET /attempts/{id}` restores questions, answers, remaining time	Resumes	Time derived from `deadline_at`	REC-04
Expired session mid-round	401	Refresh token flow; if unrecoverable, prompt re-login without losing local queue	Sign-in modal preserving route	Queue flushes after re-auth; deadline unaffected	REC-05
Application restart/deploy	—	Stateless; deadlines in DB; DB migrations backward compatible	Possibly one retried request	No deployments in the event freeze window without approval (RULES §11)	REC-06
Serverless cold start/throttle (`503 FUNCTION_THROTTLED` `[V]`)	503	Client retry with jitter; pre-warm	Slight delay	Watch burst limit; paid plan	REC-07
Interrupted export	Job heartbeat missing	Resume from cursor; parts persisted	"Export resuming…"	Retries ⩽ 3, then `failed` with reason	REC-08
Failed background job	`jobs.attempts`, `last_error`	Exponential backoff; dead-letter status; alert	Delayed results/emails	Admin "run now"; idempotent handlers	REC-09
Sweeper missed	Attempts past deadline still `in_progress`	Lazy finalization on next read/write and at round close	None	Round finalize blocks until sweep completes	REC-10
Redis/limiter outage	Client errors	Participant paths fail-open with local limiter; auth/export fail-closed	None or slower auth	Alert; budget cap	REC-11
Email provider cap reached	Provider 4xx/429	Queue holds emails; sends within daily cap	Delayed verification	Optional verification; paid tier	REC-12
Provider pause (Supabase Free inactivity)	Cannot connect	Health probe fails	Site errors	Never rely on Free for events; keep-alive on staging; runbook	REC-13
Clock skew (client)	Offset estimate	Display only	Countdown may differ slightly	Server is authoritative	REC-14

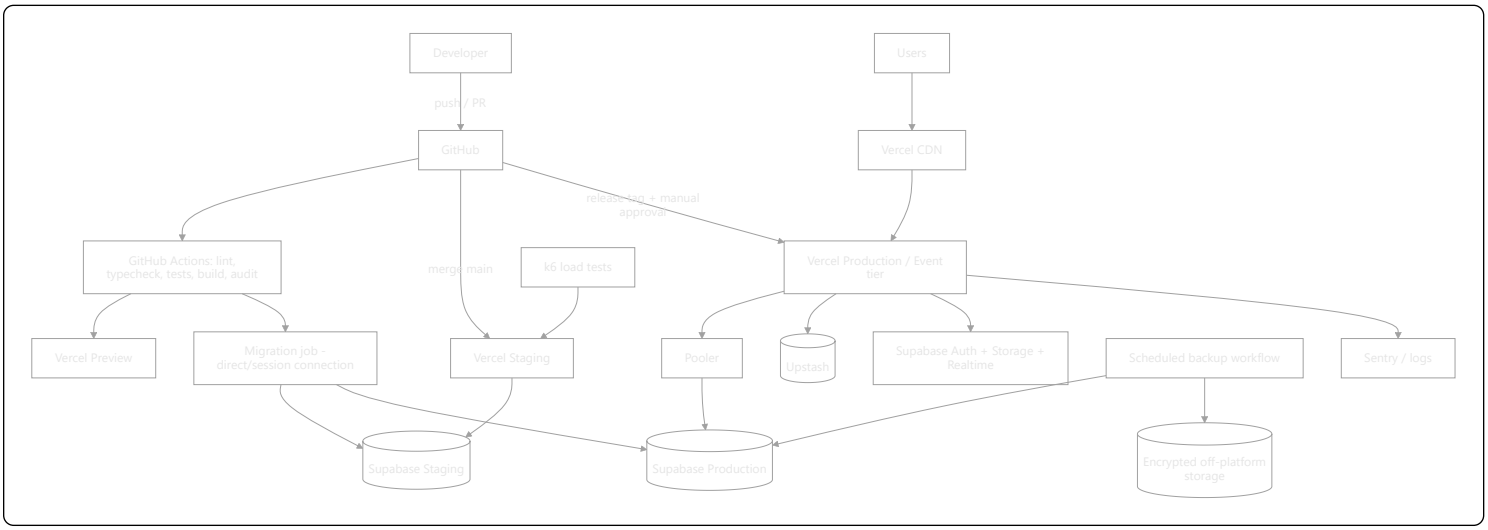
Timed rounds and restarts (NFR-REL-03): deadlines are absolute `timestamptz` values computed **once** at attempt/challenge start using the database clock; pausing adds recorded extensions (BR-14); nothing depends on process memory, cron accuracy, or the app server's clock.

21. Deployment, environments and configuration

21.1 Environments

Environment	Hosting	Database	Purpose
Local	`next dev`; local Postgres or Supabase local stack `[U]`	Local	Development, seeds
Preview	Vercel preview per PR	Shared staging or ephemeral DB `[P]`	Review, e2e smoke
Staging	Vercel (Hobby if permitted)	Supabase project #1 (Free) — **must match event-tier region/compute during load tests**	Integration, rehearsals, load tests
Production (regular)	Vercel	Supabase project #2	Ordinary operation
Event tier	Vercel Pro `[P]`	Supabase Pro, Small+ compute	Live events (§4.5)

21.2 Deployment architecture



21.3 CI/CD pipeline and release procedure

1. **PR:** lint → typecheck → unit → integration (real Postgres service container) → build → dependency/secret scan → Playwright smoke on preview → required review.
2. **Merge to `main`:** deploy staging; run **expand-only** migrations first; smoke test; k6 smoke.
3. **Release:** tag → manual approval → backup → migrations (expand) → deploy → post-deploy smoke → monitor. Contract migrations ship in a later release.
4. **Rollback:** promote previous Vercel deployment; DB rollback only by forward-fix or restore (migrations are not auto-reversed).
5. **Event freeze:** no production deployments from T−2 h to event close except approved hotfixes (RULES §11).

21.4 Environment variables (names only)

Variable	Scope	Purpose
`APP_ENV`	server	`local` / `preview` / `staging` / `production`
`NEXT_PUBLIC_APP_URL`	client	Canonical origin (CSRF/origin checks, links)
`LOG_LEVEL`	server	Logging verbosity
`NEXT_PUBLIC_SUPABASE_URL`	client	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	client	Public client key (name may differ by Supabase key generation `[U]`)
`SUPABASE_SERVICE_ROLE_KEY`	**server only**	Auth admin, Storage, Realtime broadcast — never in the browser
`DATABASE_URL`	server	Pooled (transaction mode) runtime connection, role `app_rw`
`DATABASE_READONLY_URL`	server	Pooled connection, role `app_ro` (exports/reports)
`DATABASE_MIGRATE_URL`	CI only	Direct/session connection for migrations (owner role)
`DB_POOL_MAX`	server	Per-instance pool size (default 2)
`DB_STATEMENT_TIMEOUT_MS`	server	Default statement timeout
`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`	server	Rate limiter
`RATE_LIMIT_ENABLED`	server	Kill switch (never disabled in production)
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`	server	App-sent email (Supabase Auth SMTP is configured in its dashboard with the same provider)
`EMAIL_DAILY_CAP`	server	Sender queue honors provider cap
`CRON_SECRET`	server	Bearer secret for `/api/v1/internal/jobs/tick`
`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	client/server	Optional CAPTCHA `[P]`
`CSP_REPORT_URI`	server	CSP violation reports
`SENTRY_DSN`	server/client	Error tracking `[U]`
`SENTRY_AUTH_TOKEN`	CI	Source-map upload
`ALERT_WEBHOOK_URL`	server/CI	Alert channel `[P]`
`ATTEMPT_SAVE_GRACE_MS`, `ATTEMPT_SUBMIT_GRACE_MS`	server	Default grace (3000 / 0) `[T]`
`LB_MIN_REBUILD_MS`	server	Leaderboard rebuild debounce (5000) `[T]`
`EXPORT_PACING_MS`, `EXPORT_CHUNK_ROWS`, `EXPORT_FAST_PATH_ROWS`	server	Export tuning `[T]`
`FEATURE_REALTIME_PARTICIPANTS`, `FEATURE_EXPORT_BOM`	server/client	Feature flags
`BACKUP_DATABASE_URL`, `BACKUP_AGE_PUBLIC_KEY`, `BACKUP_S3_ENDPOINT`, `BACKUP_S3_BUCKET`, `BACKUP_S3_ACCESS_KEY_ID`, `BACKUP_S3_SECRET_ACCESS_KEY`	CI only	Off-platform encrypted backups
`K6_BASE_URL`, `K6_ADMIN_TOKEN`, `K6_PARTICIPANT_TOKENS_FILE`	load-test only	k6 configuration

21.5 `.env.example` (safe template — no real values)

```dotenv
```

```
# ===== App =====
APP_ENV=local
NEXT_PUBLIC_APP_URL=http://localhost:3000
LOG_LEVEL=info

# ===== Supabase (Auth, Storage, Realtime) =====
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
# SERVER ONLY - never expose to the browser or commit a real value
SUPABASE_SERVICE_ROLE_KEY=

# ===== Database (pooled = transaction mode) =====
DATABASE_URL=
DATABASE_READONLY_URL=
# CI / migrations only (direct or session-mode connection)
DATABASE_MIGRATE_URL=
DB_POOL_MAX=2
DB_STATEMENT_TIMEOUT_MS=5000

# ===== Rate limiting (Upstash Redis) =====
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
RATE_LIMIT_ENABLED=true

# ===== Email (SMTP provider) =====
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
EMAIL_FROM=
EMAIL_DAILY_CAP=300

# ===== Jobs =====
CRON_SECRET=

# ===== Security (optional) =====
TURNSTILE_SITE_KEY=
TURNSTILE_SECRET_KEY=
CSP_REPORT_URI=

# ===== Observability =====
SENTRY_DSN=
SENTRY_AUTH_TOKEN=
ALERT_WEBHOOK_URL=

# ===== Tunables (defaults are targets, not measured values) =====
ATTEMPT_SAVE_GRACE_MS=3000
ATTEMPT_SUBMIT_GRACE_MS=0
LB_MIN_REBUILD_MS=5000
EXPORT_PACING_MS=150
EXPORT_CHUNK_ROWS=1000
EXPORT_FAST_PATH_ROWS=5000

# ===== Feature flags =====
FEATURE_REALTIME_PARTICIPANTS=false
FEATURE_EXPORT_BOM=true

# ===== CI-only secrets (define in GitHub Actions secrets, not in the app) =====
# BACKUP_DATABASE_URL=
# BACKUP_AGE_PUBLIC_KEY=
# BACKUP_S3_ENDPOINT=
# BACKUP_S3_BUCKET=
```

```
# BACKUP_S3_ACCESS_KEY_ID=
# BACKUP_S3_SECRET_ACCESS_KEY=

# ===== Load tests (local/CI only) =====
# K6_BASE_URL=
# K6_ADMIN_TOKEN=
# K6_PARTICIPANT_TOKENS_FILE=
```

22. Monitoring

Signal	Source	Alert (targets `[T]`)
Availability	Uptime pings on `/api/health` (shallow) and a rate-limited deep check with a trivial DB query	2 consecutive failures
Error rate	Sentry `[U]` + structured logs	5xx > 1% over 5 min
Latency	Request timing logs by route class	Save/submit p95 above target for 5 min
DB saturation	Supabase reports: CPU, memory, disk IOPS, connections, pooler clients `[V: available on Free and Pro]`	CPU > 80%, pooler clients > 70% of cap
Job health	`jobs` table: oldest `queued` age, failures	Queue age > 60 s live / > 10 min otherwise
Overdue attempts	Count of `in_progress` past deadline + grace	> 0 for > 60 s
Rate-limit hits	Redis counters/logs	Sudden spike on auth/registration
Email queue	Pending count vs daily cap	> 80% of cap
Export health	Running/failed jobs	Any failed
Quotas	Provider usage pages (Vercel, Supabase, Upstash)	Any quota > 70%
Business	Attempts started/submitted, submissions, evaluations pending, leaderboard version/freshness	Freshness > 30 s live

Structured log fields: `ts, level, request_id, route, method, status, latency_ms, event_id, round_id, actor_hash, error_code`. **Event-day dashboard:** live attempts, saves/s, submit/s, error rate, DB CPU, pooler clients, job queue age, leaderboard version age. Log retention is limited on free plans `[V: Supabase 1 day on Free; Vercel Hobby log retention reported ≈ 1 h ]`[S]`]` — production/event tier must ship logs to a durable sink `[P]`.

23. Backup and recovery

Item	Design (targets `[T]`/`[P]`)
Provider backups	Supabase Free: **none** `[V]`; Pro: daily backups retained 7 days `[V]`; PITR optional at $100/month per 7 days `[V]`
Own logical backups	GitHub Actions scheduled `pg_dump` of the app schema (and required Auth data where permitted `[U]`), compressed and **encrypted with** `age`, uploaded off-platform; daily, plus **on-demand before and after every round** and before every release/migration
Retention	Daily × 14, per-round × event lifetime + 90 days `[P]`
Storage objects	Question images/banners mirrored to the backup bucket after change `[U]`
RPO target	Event day: ⩽ 1 round of data if the database is destroyed (provider PITR not assumed); otherwise committed writes are durable at ack. Owner may buy PITR to reduce RPO `[P]`
RTO target	⩽ 60 min to restore into a fresh project and repoint config `[T]`
Restore drill	Monthly and before the first live event: restore latest dump into an empty project, run integrity checks (`scripts/verify-restore`), compare row counts and checksums, run smoke tests. **A backup is not considered valid until a restore has been rehearsed**
Point-of-no-return safeguards	Event archive export (results, responses, evaluations CSV + JSON manifest) generated at completion
Degraded mode	Public leaderboard/results snapshot cacheable at the CDN with stale-if-error `[U]`; static "results as of" export can be posted if the DB is unavailable

24. Testing and load-test strategy

24.1 Test layers

Layer	Tooling	Scope / must-cover
Unit	Vitest	`scoreAttempt`, `computeResults`, tie-break/ranking, aggregation, PRNG shuffle determinism, URL validator, CSV sanitizer (injection corpus), time/deadline math, permission function
Integration	Vitest + real Postgres (service container)	Transactions and constraints: capacity race, participant counter, idempotent submit, `clientSeq` upsert, `FOR UPDATE` submit, advisory-lock single-flight, append-only triggers, audit same-transaction, keyset export, RLS deny-by-default
Contract/API	Vitest + supertest-style	Every route: authz matrix, Zod rejection, error shape, idempotency replay
End-to-end	Playwright	Register → team → take quiz → submit challenge → judge → publish → leaderboard → export; mobile viewport; keyboard navigation
Security	Custom + tools	Authz matrix per role, cross-event isolation (SEC-ISO), IDOR attempts, CSRF/origin checks, XSS payloads in descriptions/comments, CSV injection, rate-limit behavior, SSRF-style URLs, secret scan, dependency audit
Recovery	Scripts + fault injection	REC-01…REC-14 (§20): DB disconnect, restart mid-attempt, duplicate delivery, sweeper disabled, export kill/resume
Accessibility	axe + manual	WCAG 2.2 AA checks on key flows
Performance	Lighthouse CI; k6	Budgets in DESIGN.md; load scenarios below

24.2 Load-test environment

- Dedicated staging stack **identical to the event tier** (same Supabase plan/compute, region, Vercel plan) — results on Free tiers are not evidence for the event tier.
- Realistic seed: 1,000 users, 500+ approved participants, a 30-question quiz, a challenge round, a judged round, 10 staff accounts.
- Auth: **pre-mint sessions** for virtual users to avoid measuring Auth rate limits and email caps; test the login path separately at lower volume.
- Load generators: k6 open-source runners and/or Grafana Cloud k6 (Free 500 VUh/month `[V]`; per-test VU cap on Free `[U]`) from ⩾ 2 locations `[U]`; generator CPU and network are monitored so the generator is not the bottleneck.
- After each test: **reconciliation** — client-side acknowledged writes vs database rows (must match exactly), plus check for duplicate attempts/scores.

24.3 Scenarios

ID	Scenario	Profile	Pass criteria [T]
LT-0	Smoke	5 VUs, 2 min	0 errors
LT-1	Browse + dashboard + login mix	Ramp 100 → 250 → 500 VUs, 10 min each	Page/API p95 within §18.5; 5xx < 0.5%
LT-2	**Simultaneous quiz start**	500 starts within 30 s	Start p95 ⩽ 1.2 s; no duplicate attempts; pooler < 70%
LT-3	**Sustained answer saving**	500 VUs, saves every 5–40 s for 30 min (and a compressed 10 min run)	Save p95 ⩽ 500 ms / p99 ⩽ 1.5 s; 0 lost
LT-4	**Concentrated submit surge**	500 submits in 3 s; variant with 0–3 s jitter	Submit p95 ⩽ 1 s / p99 ⩽ 2.5 s; exactly one score per attempt
LT-5	**Leaderboard viewers**	500 pollers (15 s, ETag) while LT-4 runs	Read p95 ⩽ 300 ms; freshness ⩽ 15 s p95; exactly one rebuild per interval (lock works)
LT-6	**Synchronized challenge deadline**	250–500 submissions in 60 s incl. 10% duplicates/retries	Submit p95 ⩽ 1 s; revisions consistent; late policy correct
LT-7	**Mixed live event**	500 quiz + 500 leaderboard + 10 admin + export of 10k rows	All above hold; export ⩽ 60 s
LT-8	**Headroom**	Step 500 → 750 → 1,000 VUs	Identify knee; ⩾ 50% headroom desirable
LT-9	**Soak**	250 VUs, 60 min	No latency drift, memory/connection leaks
LT-10	**DB pressure & recovery**	Slow query injection, connection kill, pooler saturation	Graceful 503s; recovery without data loss
LT-11	**Registration surge**	500 registrations/60 s, capacity 400	Exactly 400 accepted; rest waitlisted/rejected; unique codes

24.4 Reporting

Each run records: build SHA, config (plan/compute/region), scenario, VUs, duration, throughput (rps), latency p50/p95/p99 per route, error rates by class, DB CPU/memory/IOPS/connections, pooler clients, function throttles, reconciliation result, pass/fail per criterion, and links to raw output. **Status today: no load test has been run; all figures are targets.** Claims of readiness require an attached report (RULES §10).

25. Decision register

Status **Proposed** = recommended in this document, **not approved**. Nothing below is final.

ID	Decision	Status	Alternatives	Decide by
AD-01	Modular monolith (Next.js + TS)	Proposed	Separate API service; Workers	M0
AD-02	Supabase Postgres as system of record	Proposed	Neon; D1	M0
AD-03	Supabase Auth behind `AuthProvider`	Proposed	Auth.js; custom	M0
AD-04	Drizzle ORM + SQL migrations	Proposed	Prisma; Kysely	M1
AD-05	Polling snapshots baseline; realtime ping optional	Proposed	WebSocket server; Ably	M0
AD-06	Upstash Redis for rate limiting	Proposed	Postgres counters; edge firewall only	M0
AD-07	DB-backed jobs + tick endpoints	Proposed	BullMQ; pg_cron only	M1
AD-08	Scores as scaled integers/numeric	Proposed	Floats	M1
AD-09	Live leaderboard from snapshots; official results from result runs	Proposed	Only official runs	M0
AD-10	Export as chunked resumable parts	Proposed	Single streaming response only	M11
AD-11	Event tier on paid plans; free tiers for dev/staging	Proposed — needs owner approval (D-P02)	Free-only with accepted risk	M0
AD-12	Region co-location (Mumbai if available)	Pending verification `[U]`	Nearest available	M0
AD-13	Own encrypted off-platform backups	Proposed	Provider backups only	M1
AD-14	Tailwind + Radix primitives design system	Proposed	Other component libs	M2
AD-15	Quiz question payload cache per round	Proposed, optional	None	M13

26. Extensibility and migration

Change	Approach
New round type (e.g., coding auto-judge, file upload, buzzer)	Implement `RoundType` (§10.1), add tables if needed, register; scoring integrates through `RawScoreInput` → `score_records`; export columns via `exportColumns`
New export type	Register a serializer + column definition; reuse job runner and sanitizer
New registration field type	Add to field type registry with validator and CSV formatter
Multi-organization	Add `organization_id` above `events` and to role scopes; composite FKs already isolate by event
Auth provider swap	Implement `AuthProvider`; migrate identities via `users.auth_user_id`; password migration requires reset unless hashes are exportable
DB provider swap	Standard Postgres dump/restore; re-point `DATABASE_*`; verify extensions
Scale beyond MVP	Read replicas (offered on higher Supabase tiers `[S]`), partition `quiz_responses`/`audit_logs` by month, dedicated realtime for participants
Config evolution	`type_config_version` + `migrateConfig`; scoring configs immutable and versioned

27. Known limitations and open items

1. No repository was available; no code exists (§2).
2. All performance figures are targets; the free-tier-only path is unverified and documented as unsuitable for a live 500-participant event (§4.4).
3. Unverified items `[U]` listed in §4.2 must be resolved in T-0.5 before commitments (GitHub Actions minutes, Sentry, Vercel Cron/Firewall, `pg_cron`, regions, Turnstile).

4. Vercel Hobby commercial-use eligibility depends on the event's nature (D-P02).
5. Cheating (shared answers, multiple accounts, external help) cannot be fully prevented by a web app; the platform provides rate limits, audit trails and duplicate-link flags, not guarantees.
6. GitHub ownership is not verified (BR-18).
7. Team-quiz semantics, grace periods, missing-round policy and privacy retention are pending approval (PRD §11).
8. Supabase Auth email limits and custom-SMTP ramp-up behavior need confirmation on the official page [S].
9. Realtime for participants is deferred; only staff realtime is planned for the MVP.
10. Point-in-time recovery is not included in the base plan; RPO depends on the backup cadence unless PITR is purchased.